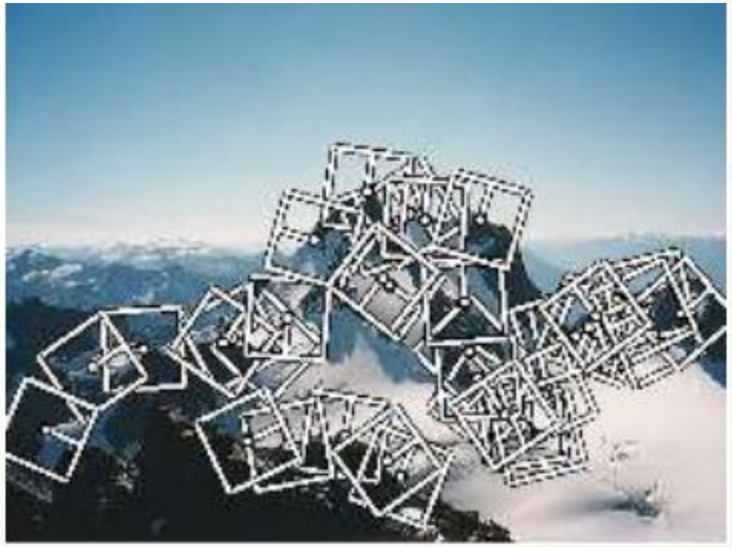
Feature Detection and Matching

Chapter 4

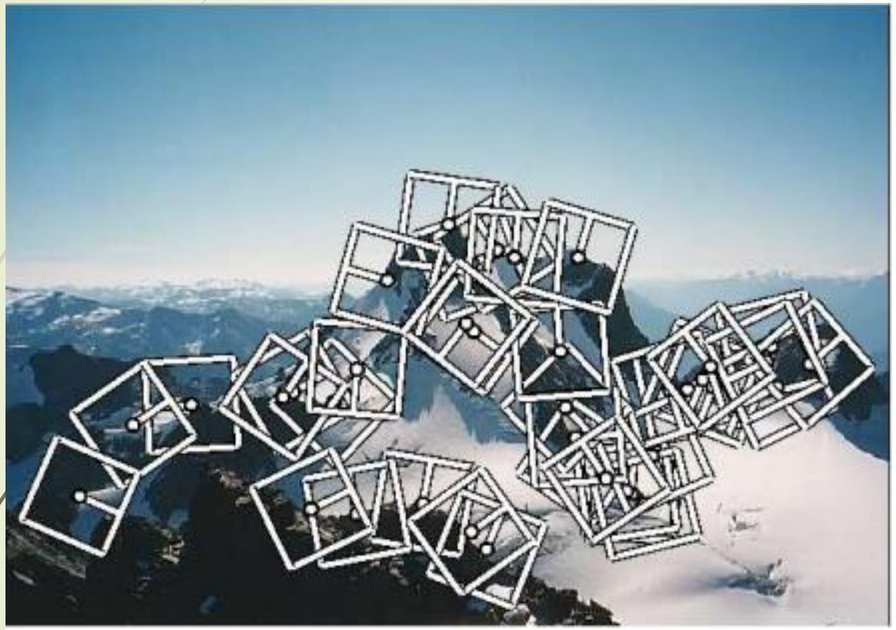


Contents

Introduction

- Feature detection is a computer vision technique that identifies and extracts unique characteristics from images.
- These characteristics, or features, are used to describe and differentiate objects in the image.
- How it works:
  - **Preprocess**: The image is enhanced to reduce noise.
  - **Detect**: Algorithms identify points of interest, such as corners or blobs.
  - **Describe**: The features are mathematically described to create a feature descriptor
- It is used in many applications, including: **object recognition, image matching, motion tracking and scene understanding**.

Points and Patches



(a) point-like interest operators

(b) region-like interest operators

Figure: A variety of feature detectors and descriptors can be used to analyze, describe and match images

Points and Patches

(c) Edges

(d) Straight lines

Figure: A variety of feature detectors and descriptors can be used to analyze, describe and match images

Points and Patches

Keypoint features or interest points

- The specific locations in the images are keypoint features or interest points.
  - mountain peaks
  - building corners
  - doorways
  - interestingly shaped patches of snow

Edges

- These kinds of features can be matched based on their orientation and local appearance (edge profiles).
- And it can also be good indicators of object boundaries and occlusion events in image sequences.
- Edges can be grouped into longer curves and straight line segments, which can be directly matched or analyzed to vanishing points.

Points and Patches

Main component of feature detection and matching

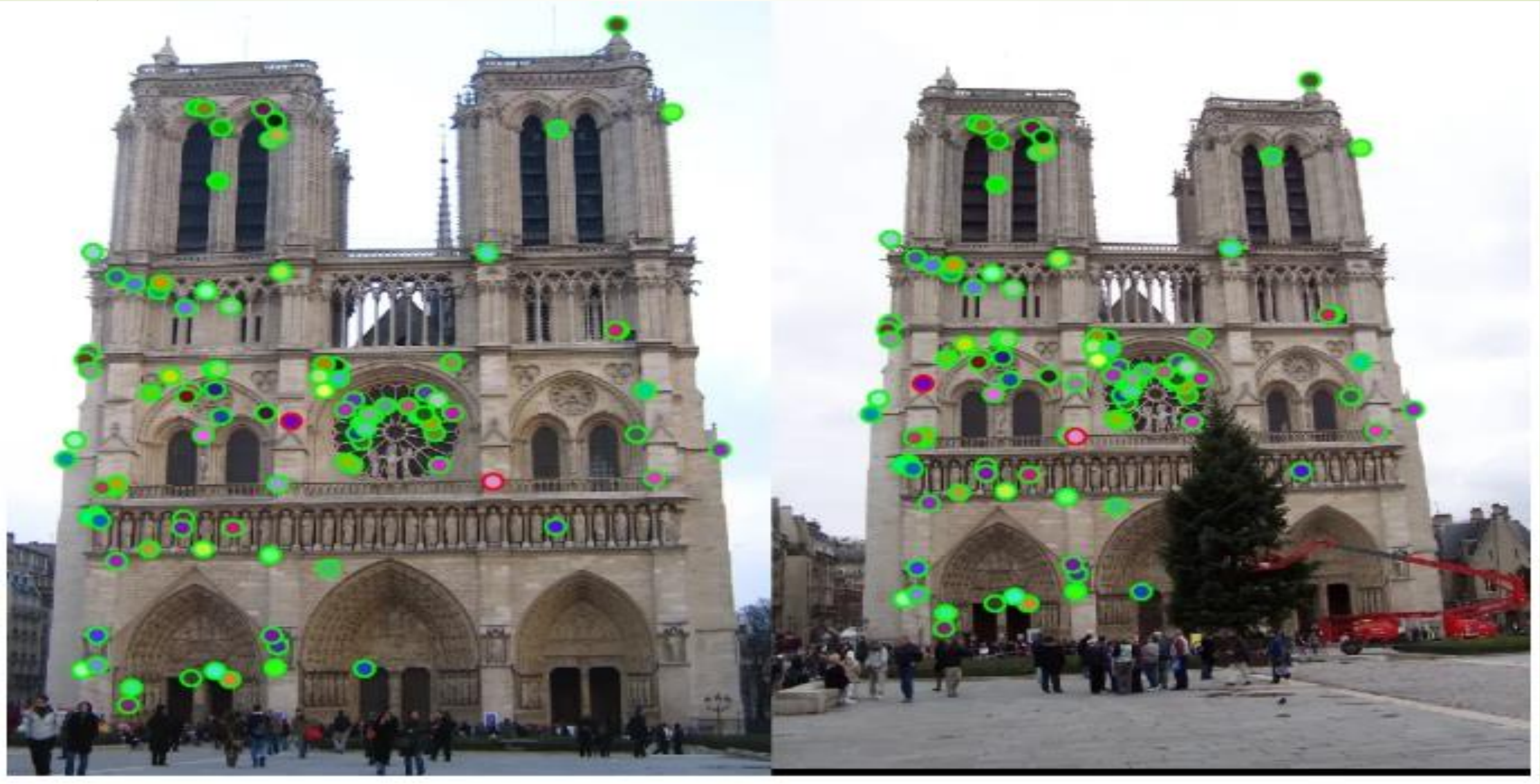
- Detection: Identify the Interest Point
- Description: It ends up with a descriptor vector for each feature point.
- Matching: Descriptors are compared across the images, to identify similar features.

Interest point

- It is the point at which the direction of the boundary of the object changes abruptly or intersection point between two or more edge segments.
- *Possible Approaches:*
  - Based on the brightness of an image(Usually by image derivative).
  - Based on Boundary extraction(Usually by Edge detection and Curvature analysis).
- *Algorithms for Identification*
  - Harris Corner, SIFT (Scale Invariant Feature Transform), SURF (Speeded Up Robust Feature), FAST (Features from Accelerated Segment Test), ORB (Oriented FAST and Rotated BRIER)

Points and Patches

Interest Points



Points and Patches

Two main approaches to finding feature points and their correspondences

- To find features in one image that can be accurately ***tracked*** using a local search technique, such as correlation or least squares.
  - More suitable when images are taken from nearby viewpoints or in rapid succession (e.g., video sequences)
- To independently detect features in all the images under consideration and ***match*** features based on their local appearance.
  - More suitable when a large amount of motion or appearance change is expected (e.g., in stitching together panoramas)

Points and Patches

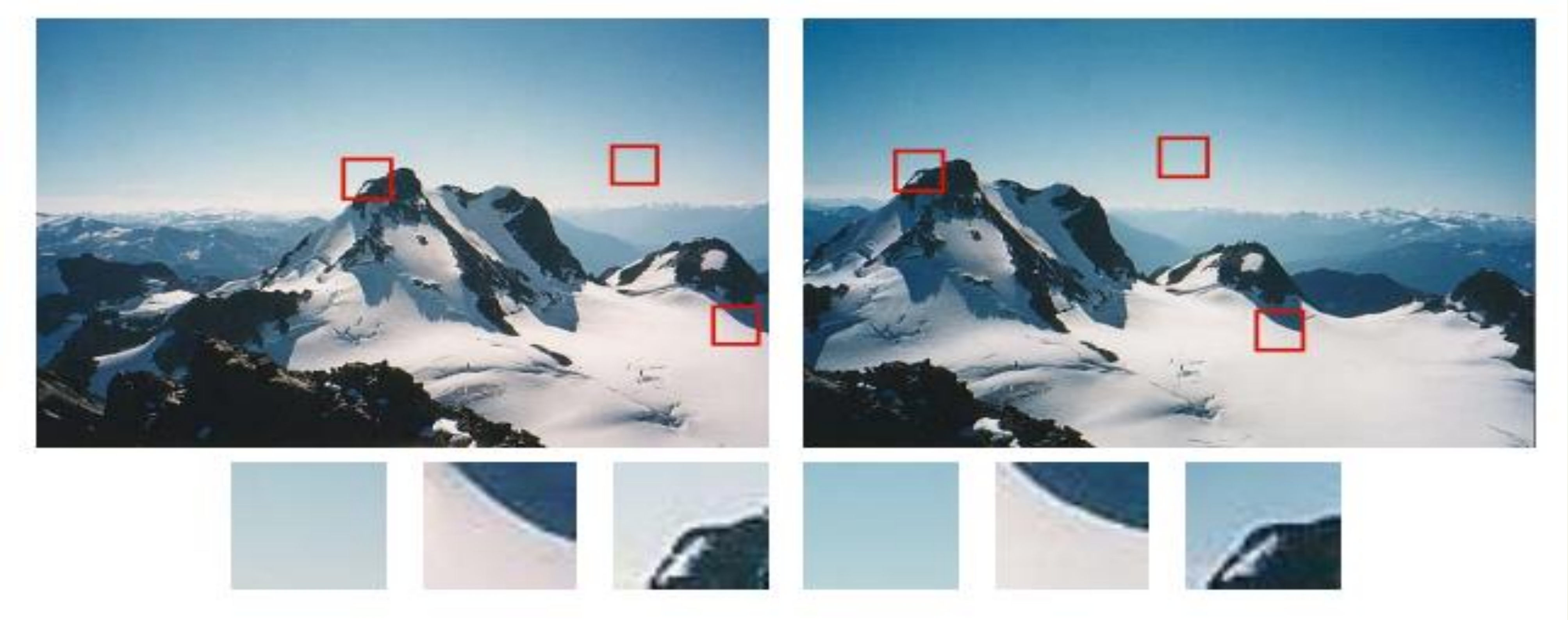
Keypoint Detection and Matching Pipeline

- Feature detection (extraction) stage : each image is searched for locations that are likely to match well in other images.
- Feature description stage : each region around detected keypoint locations is converted into a more compact and stable (invariant) descriptor that can be matched against other descriptors.
- Feature matching stage : efficiently searches for likely matching candidates in other images.
- Feature tracking stage : an alternative to the third stage that only searches a small neighborhood around each detected feature and is therefore more suitable for video processing.

Points and Patches

Feature Detector

- Texture less patches are nearly impossible to localize.
- Patches with large contrast changes (gradients) are easier to localize

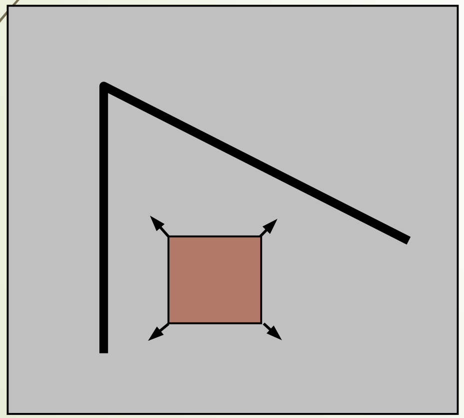


Points and Patches

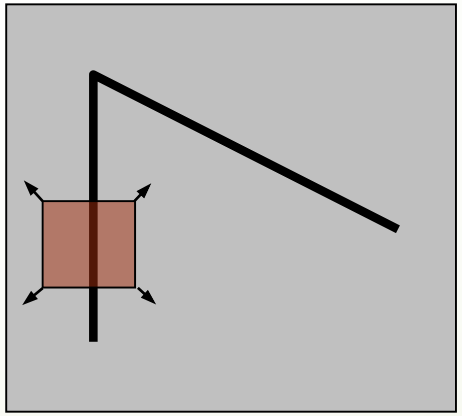
Feature Detector

Local Measure of Feature Uniqueness

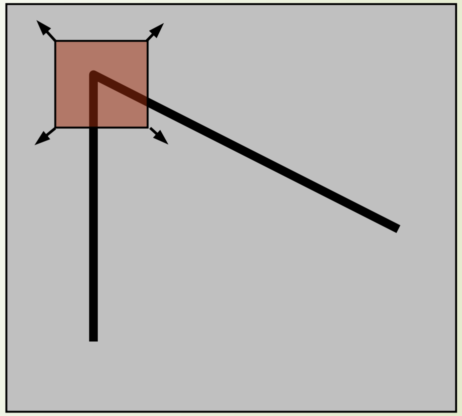
- How does the window change when you shift it?
- Shifting the window in any direction causes a big change



"flat" region:
no change in all directions



"edge":
no change along the edge direction



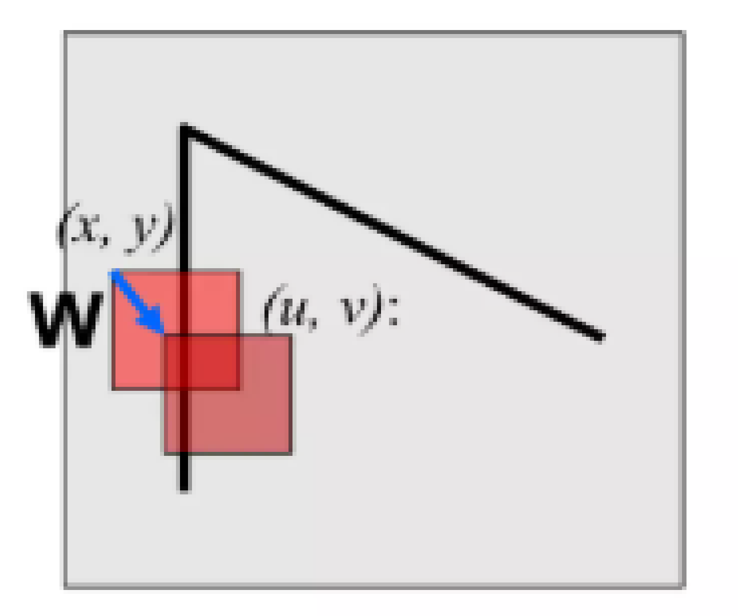
"corner":
significant change in all directions

Points and Patches

Feature Detector

- Shifting the window W by (u,v)
- Compare each pixel before and after shift image by ***Summing the Squared Differences (SSD)***

$$E_{WSSD}(u,v) = \sum_i \boldsymbol{w(x_i, yi)[I_0(x_i + u, yi + v) - I_0(x_i, yi)]^2}$$



I_0 and I_1 are the two images being compared

(u,v) is the displacement vecor

w(x,y) is a spatially varying weighting function

the summation *i* is over all the pixels in the patch.

Points and Patches

Feature Detector

- Auto-Correlation function or surface
- When performing feature detection, we do not know which other image locations the feature will end up being matched against.
- Therefore, we can only compute how stable this metric is with respect to small variations in position Δu by comparing an image patch against itself, which is known as an auto-correlation function or surface.

$$E_{\mathrm{AC}}(\Delta u) = \sum_i w(x_i)[I_0(x_i + \Delta u) - I_0(x_i)]^2$$

I_0: the image being compared

Δu: small variations in position

w(x): varying weighting (or window) function

summation i: over all the pixels in the patch

Points and Patches

Feature Detector

- Approximation of Auto-Correlation function or surface

$I_0(x_i + \Delta u) \approx I_0(x_i) + \nabla I_0(x_i) \cdot \Delta u$

$$\begin{aligned} E_{\mathrm{AC}}(\Delta u) &= \sum_i w(x_i)[I_0(x_i + \Delta u) - I_0(x_i)]^2 \\ &\approx \sum_i w(x_i)[I_0(x_i) + \nabla I_0(x_i) \cdot \Delta u - I_0(x_i)]^2 \\ &= \sum_i w(x_i)[\nabla I_0(x_i) \cdot \Delta u]^2 \\ &= \Delta u^T A \Delta u, \end{aligned}$$

$\nabla I_0(x_i)$: image gradient at x_i

Points and Patches

Feature Detector

- Auto-Correlation matrix

Auto-correlation matrix *A*:

$$\mathbf{A} = w * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

w: weighting kernel (from spatially varying weighting function)
I_x, I_y: horizontal and vertical derivatives of Gaussians

Points and Patches

Feature Detector

✯ Basic Feature Detection Algorithm

1. Compute the horizontal and vertical derivatives of the image I_x and I_y by convolving the original image with derivatives of Gaussians .
2. Compute the three images (I_x^2, I_y^2, I_xI_y) corresponding to the outer products of these gradients.(The matrix A)
3. Convolve each of these images with a larger Gaussian.
4. Compute a scalar interest measure using one of the formulas .
5. Find local maxima above a certain threshold and report them as detected feature point locations.

Points and Patches

Harris Detector

Let's consider a two-dimensional image I and a patch W of size $m \times m$ centered in (x_0, y_0). We want to evaluate the intensity variation occurred if the window W is shifted by a small amount (u, v). Such variation can be estimated by computing the Sum of Squared Differences (SSD):

$$\mathrm{SSD}(u, v) = \sum_{(x,y)\in W} g(x, y)\left[I(x, y) - I(x + u, y + v)\right]^2 \quad (1)$$

where $g(x, y)$ is a window function that can be a rectangular or a Gaussian function. We need to maximize the function $\mathrm{SSD}(u, v)$ for corner detection. Since u and v are small, the shifted intensity $I(x + u, y + v)$ can be approximated by the following first-order Taylor expansion:

$$I(x + u, y + v) \approx I(x, y) + uI_x(x, y) + vI_y(x, y) \quad (2)$$

where I_x and I_y are partial derivatives of I in x and y direction, respectively. By substituting (2) in (1) we obtain:

$$\mathrm{SSD}(u, v) \approx \sum_{(x,y)\in W} g(x, y)\left[u^2 I_x^2 + 2uvI_xI_y + v^2 I_y^2\right]. \quad (3)$$

The equation (3) can be expressed in the following matrix form:

$$\mathrm{SSD}(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} \quad (4)$$

where M is a 2×2 matrix computed from image derivatives:

$$M = \sum_{(x,y)\in W} g(x, y)\begin{bmatrix} I_xI_x & I_xI_y \\ I_xI_y & I_yI_y \end{bmatrix} \quad (5)$$

The matrix M is called *structure tensor*.

Points and Patches

Feature Detector

✯ Harris Detector : Steps

1. Compute Gaussian derivatives at each pixel
2. Compute auto correlation matrix in a Gaussian window around each pixel
3. Compute corner response function R
4. Threshold R
5. Find local maxima of response function

Points and Patches

Feature Detector

- Harris Coner Detector

Input Image

0	0	1	4	9
1	0	5	7	11
1	4	9	12	16
3	8	11	14	16
8	10	15	16	20

Differentiation kernels

-1	0	1	d/dx

-1	
0	d/dy
1	

Points and Patches

Feature Detector

- Harris Coner Detector Step : Compute Gaussian derivatives at each pixel

-1	0	1	**d/dx**

Input Image

0	0	1	4	9
1	0	5	7	11
1	4	9	12	16
3	8	11	14	16
8	10	15	16	20

=

Ix

x	x	x	x	x
x	4	7	6	x
x	8	8	7	x
x	8	6	5	x
x	x	x	x	x

d/dx * input image (I) = Ix

-1*1 + 0*0 + 1*5 = -1+0+5= **4**

-1*0 + 0*5 + 1*7 = -0+0+7 = **7**

-1*5+ 0*7 + 1*11 = -5+0+11 = **6**

-1*1 + 0*4 + 1*9 = -1+0+9 = **8**

-1*4 + 0*9 + 1*12 = -4 +0+12 = **8**

d/dx * input image (I) = Ix

-1*9 + 0*12 + 1*16 = -9+0+16 = **7**

-1*3+ 0*8 + 1*11 = -3+0+11 = **8**

-1*8 + 0*11 +1*14 = -8+0+14 = **6**

-1*11 + 0*14 + 1*16 = -11+0+16 = **5**

Points and Patches

Feature Detector

✯ Harris Coner Detector Step : Compute Gaussian derivatives at each pixel

-1	**d/dy**
0	
1	

Input Image

0	0	1	4	9
1	0	5	7	11
1	4	9	12	16
3	8	11	14	16
8	10	15	16	20

=

Iy

x	x	x	x	x
x	4	8	8	x
x	8	6	7	x
x	6	6	4	x
x	x	x	x	x

d/dy * input image (I) = Iy

-1*0 + 0*0 + 1*4 = 0+0+4 = **4**

-1*1 + 0*5 + 1*9 = -1+0+9 = **8**

-1*4+ 0*7 + 1*12 = -4+0+12 = **8**

-1*0 + 0*4 + 1*8 = 0+0+8 = **8**

-1*5 + 0*9 + 1*11 = -5 +0+11 = **6**

d/dy * input image (I) = Iy

-1*7 + 0*12 + 1*14 = -7+0+14 = **7**

-1*4+ 0*8 + 1*10 = -4+0+10 = **6**

-1*9 + 0*11 +1*15 = -9+0+15 = **6**

-1*12+ 0*14 + 1*16 = -12+0+16 = **4**

Points and Patches

Feature Detector

✯ Harris Coner Detector Step : Compute auto correlation matrix in a Gaussian window around each pixel

Ix

x	x	x	x	x
x	4	7	6	x
x	8	8	7	x
x	8	6	5	x
x	x	x	x	x

*

Iy

x	x	x	x	x
x	4	8	8	x
x	8	6	7	x
x	6	6	4	x
x	x	x	x	x

- $\mathbf{Ix^2}$ = 4*4 + 7*7 + 6*6 + 8*8 + 8*8 + 7*7 + 8*8 + 6*6 + 5*5 = 16+49+36+64+64+49+64+36+25 = 403
- Iy^2 = 4*4 +8*8 + 8*8 + 8*8 + 6*6 + 7*7 + 6*6 + 6*6 + 4*4 = 16+64+64+64+36+49+36+36+16 = 381
- **Ix * Iy** = 4*4 + 7*8 + 6*8 +8*8 + 8*6 + 7*7 + 8*6 + 6*6 + 5*4 = 16+56+48+64+48+49+48+36+20 = 385

M=

Ix^2	IxIy
IxIy	Iy^2

M=

403	385
385	381

Points and Patches

Feature Detector

- Harris Coner Detector Step : Compute corner response function R

  : Threshold R

M=

Ix^2	$IxIy$
$IxIy$	Iy^2

M=

403	385
385	381

*

$C = \det(M) - k\ \mathrm{trace}(M)^2$ **$(0.04 < k < 0.05)$**

$\det(M) = (Ix^2 * Iy^2) - (Ix * Iy)^2 = (403*381)-(385*385) = 153543-148225 = 5318$

$\mathrm{trace}(M)^2 = (Ix^2+Iy^2)^2 = (403+381)^2 = (784)^2 = 614656$

$C = 5318 - 0.04 * 614656 = 5318 - 24586.24 =$ **-19268.24**

If C is large : **Corner**

If C is negative : Edge

If C is small : flat

Points and Patches

Feature Detector

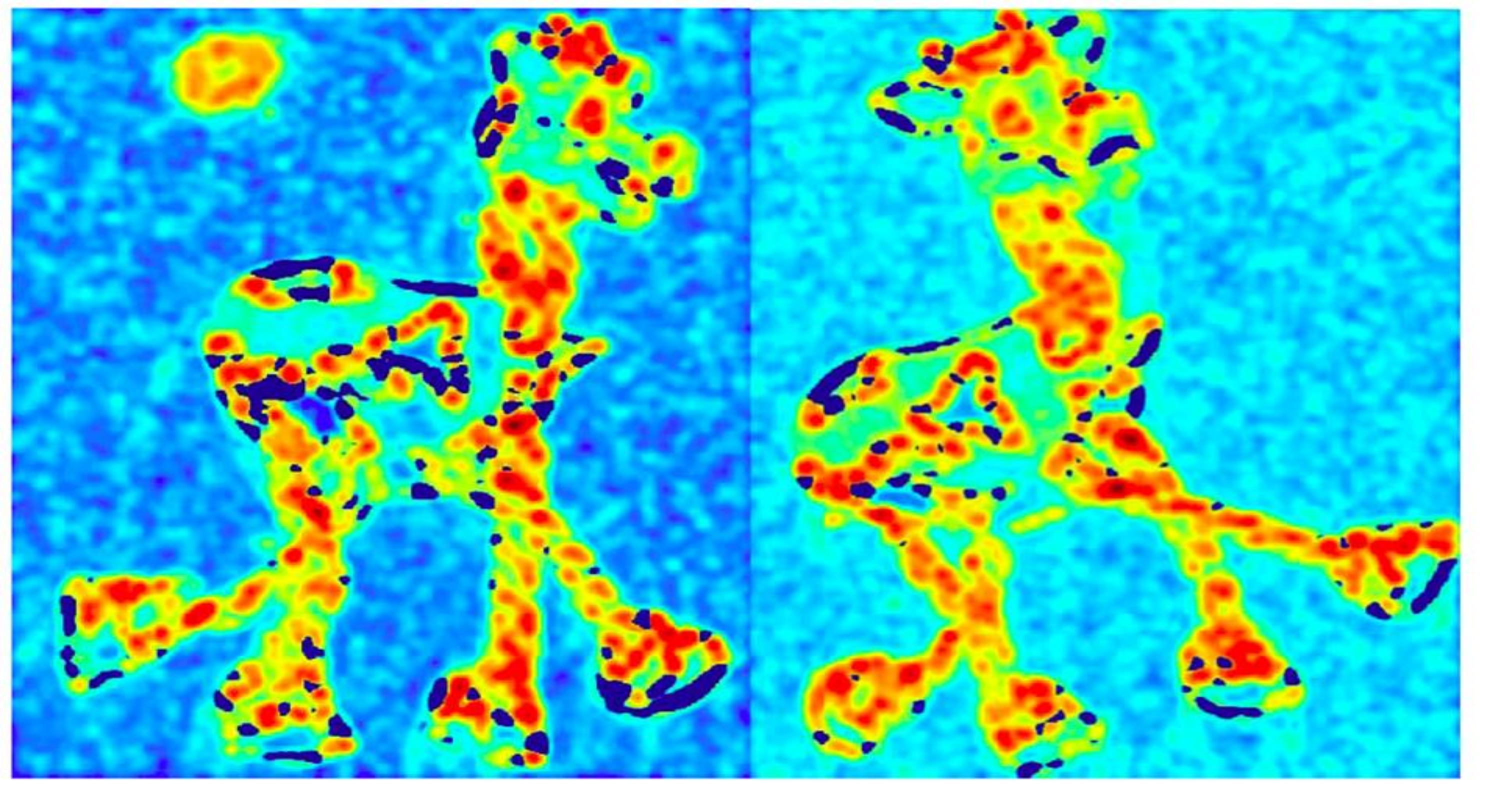
- Harris Detector : Steps
- Example

[Source: K. Grauman]

Points and Patches

Feature Detector

- Harris Detector : Steps
- Compute Corner Response R

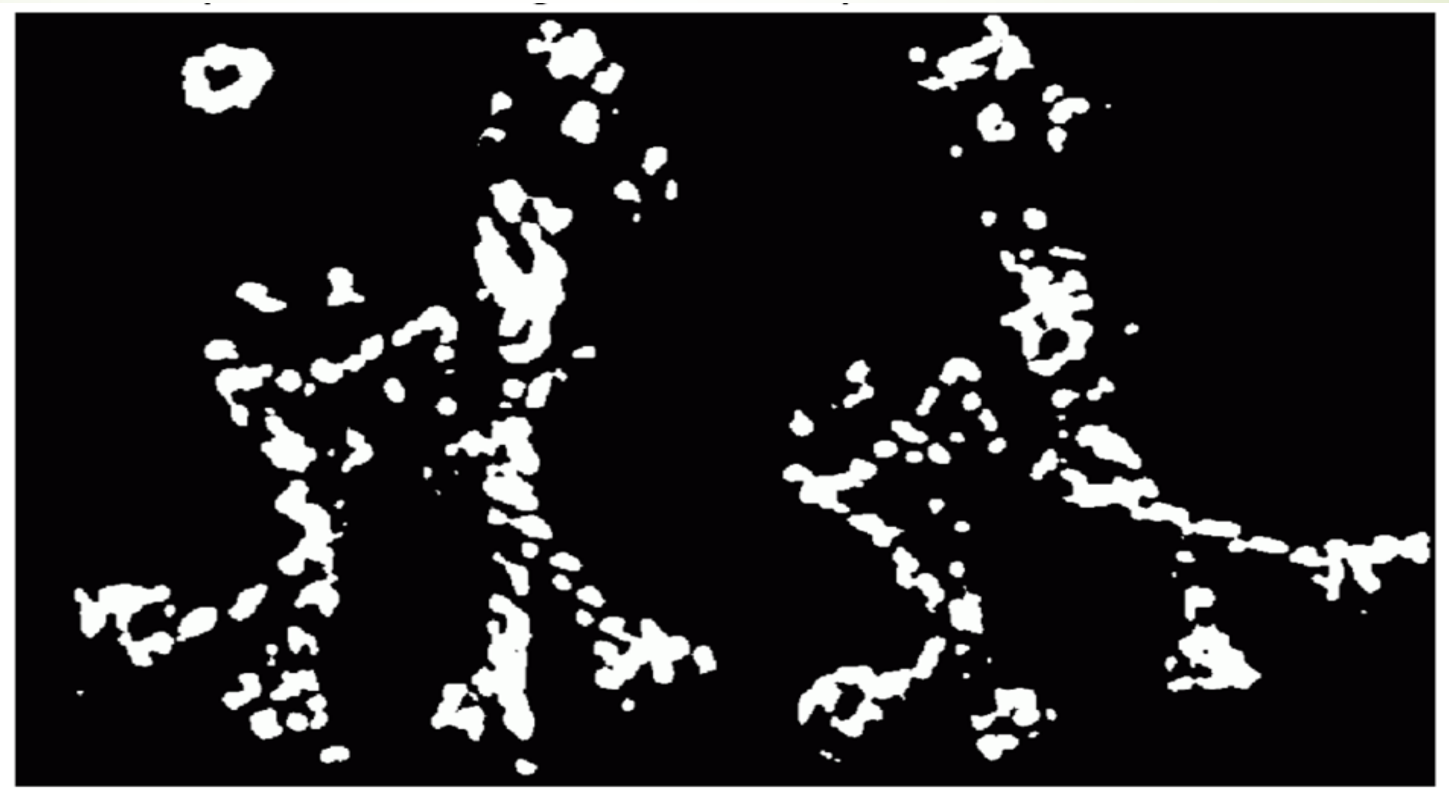


[Source: K. Grauman]

Points and Patches

Feature Detector

- Harris Detector : Steps
- Find points with large corner response: R>threshold

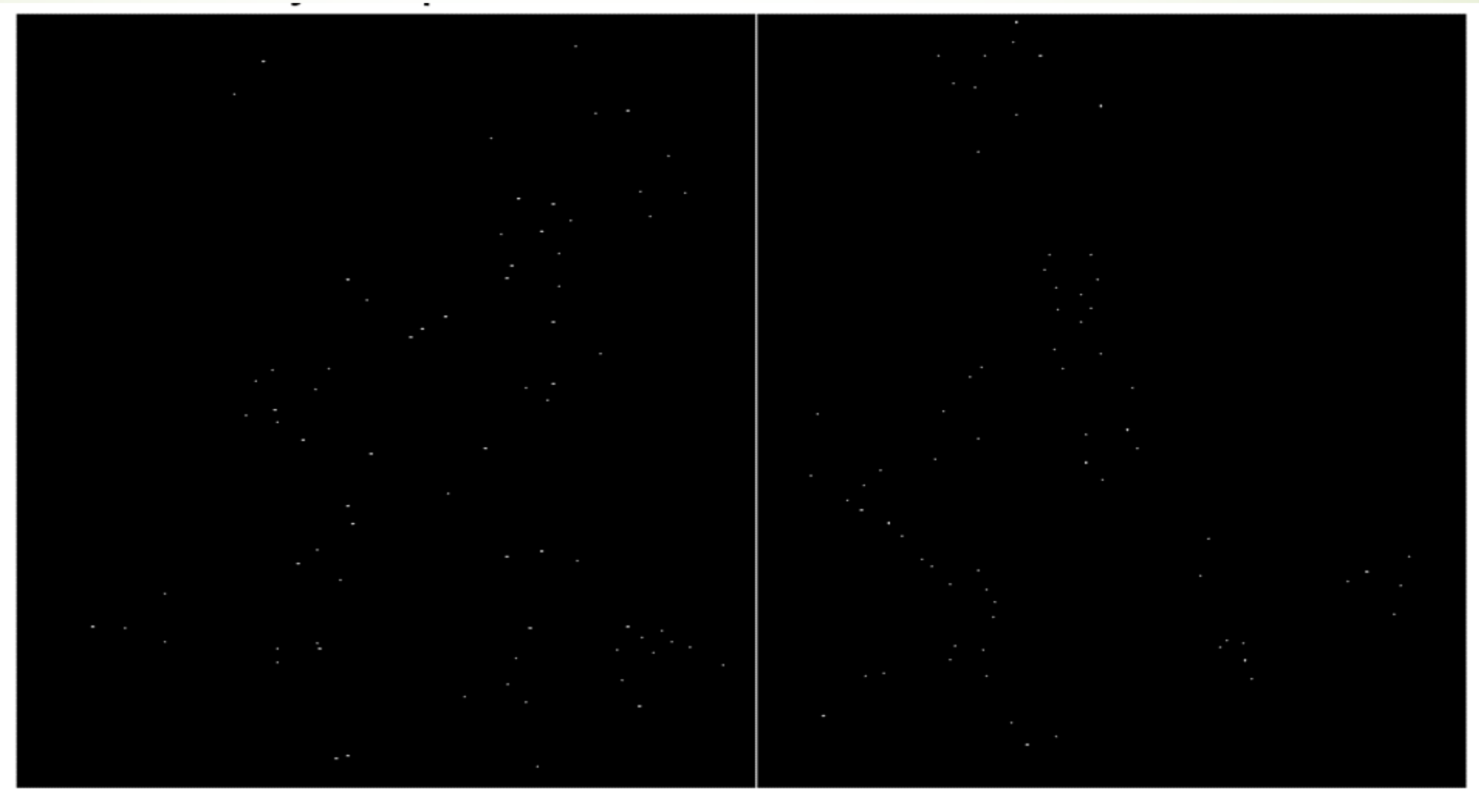


[Source: K. Grauman]

Points and Patches

Feature Detector

- Harris Detector : Steps
- Take only the points of local maxima of R

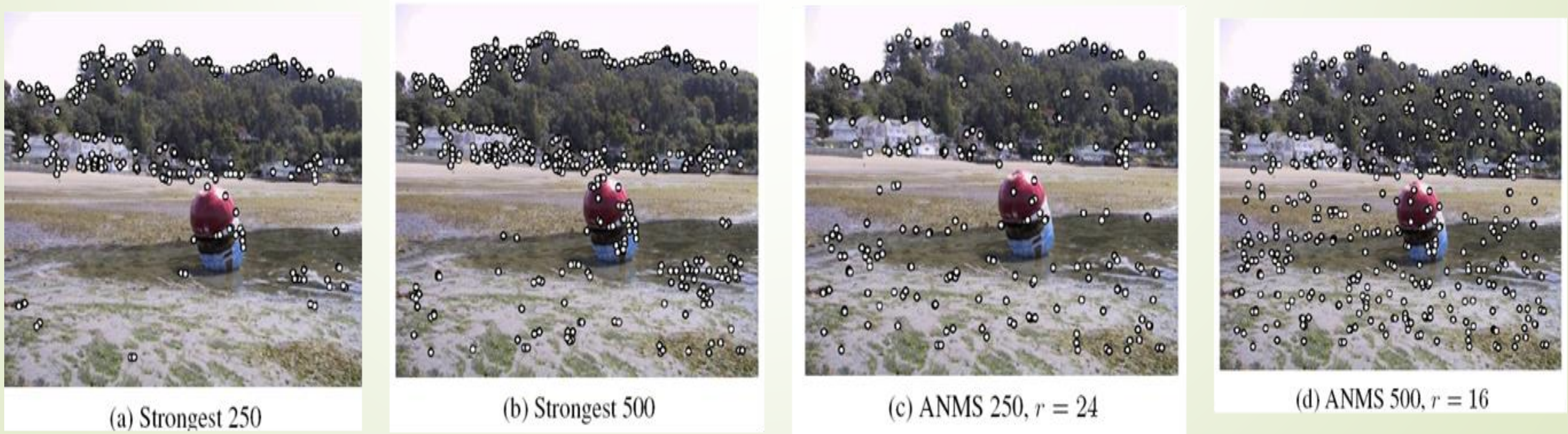


[Source: K. Grauman]

Points and Patches

Feature Detector

- Adaptive Non-Maximal Suppression (ANMS)
- ANMS is important for solving the problem of having too many interesting points in some region of the image that is denser than others.
- Detect features that are:
  - Local maxima
  - Response value is significantly (10%) greater than all of its neighbors within a radius r



(a) Strongest 250 (b) Strongest 500 (c) ANMS 250, $r = 24$ (d) ANMS 500, $r = 16$

Points and Patches

Feature Detector

- Measuring Repeatability
- Among large number of feature detectors, how to decide which ones to use?
- Measure the repeatability of feature detectors.
- Define as the frequency with which keypoints detected in one image are found within pixels of the corresponding location in a transformed image.
- It is used to assess keypoint detection performance i.e. feature detector performance.

Points and Patches

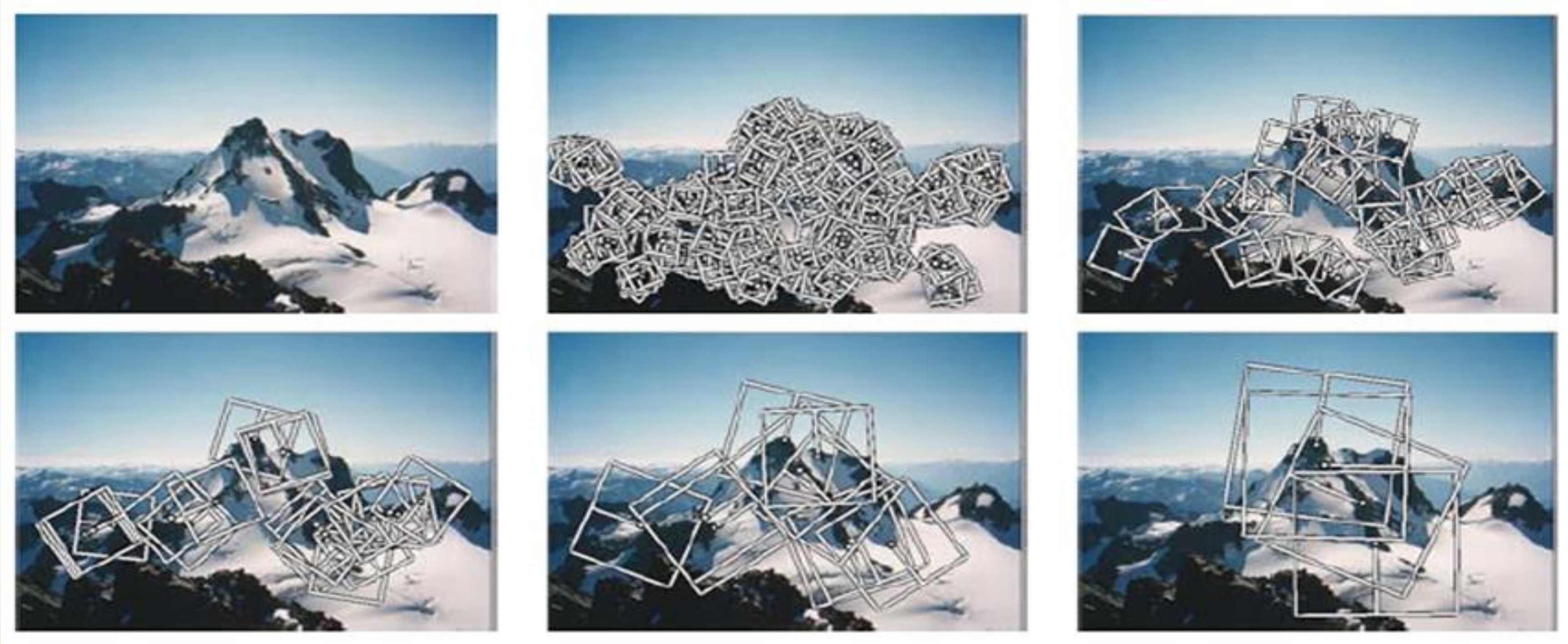
Feature Detector

- Maximally Stable Extremal Region (MSER) Detector
- Used as a method of blob detection in images.
- To find correspondences between image elements from two images with different viewpoints.
- This method of extracting a comprehensive number of corresponding image elements contributes to the wide-baseline matching.
- It has led to better stereo matching and object recognition algorithms.

Points and Patches

Feature Detector

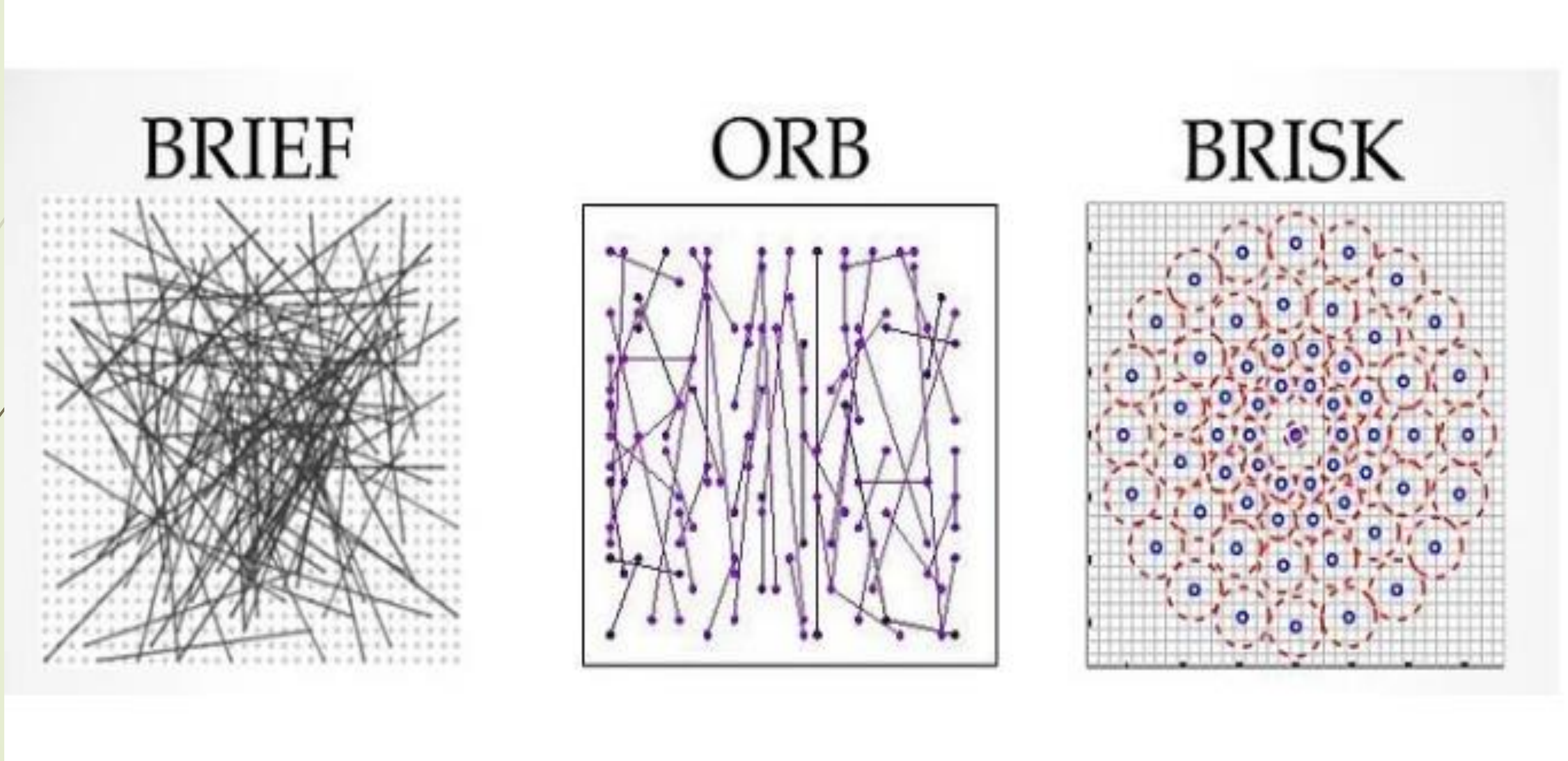
- Scale Invariance
- Problem: if no good feature points in mage?
- Solution: extract features at a variety of scale (multi-scale)
- Scale invariance is a feature of objects that do not change by varying the scales of length.



Feature Descriptor

- A feature descriptor is an algorithm which takes an image and outputs feature descriptors/feature vectors.
- Feature descriptors encode interesting information into a series of numbers and act as a sort of numerical “fingerprint” that can be used to differentiate one feature from another.
- Descriptors can be categorized into two classes:
  - **Local Descriptor:** It tries to resemble shape and appearance only in a local neighborhood around a point
  - **Global Descriptor**: A global descriptor describes the whole image.
- *Algorithms*
  - SIFT (Scale Invariant Feature Transform), SURF (Speeded Up Robust Feature), BRISK (Binary Robust Independent Elementary Features), ORB (Oriented FAST and Rotated BRIEF)

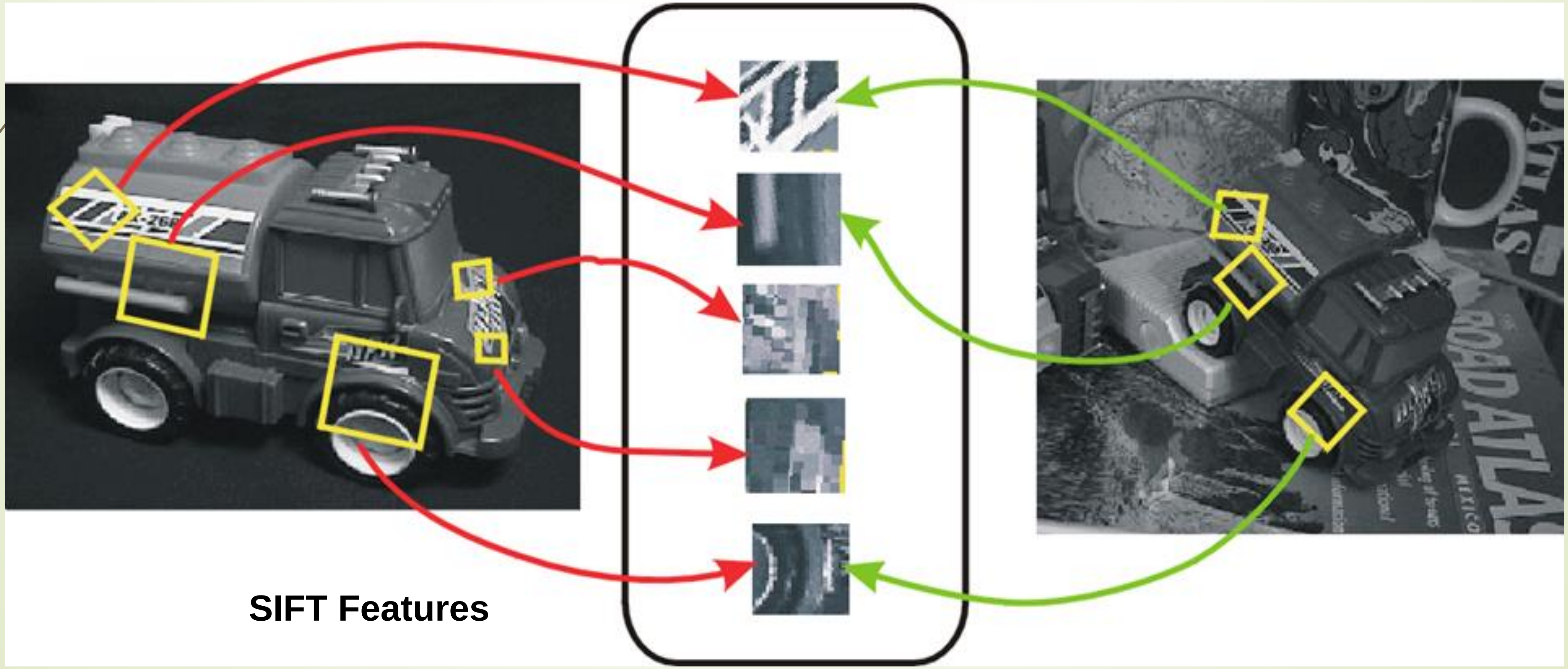
Feature Descriptor



Feature Descriptor

Scale Invariant Feature Transform(SIFT)

- It is a feature detection algorithm in computer vision to detect and describe local features in images.
- Presented in 2004, by D.Lowe, University of British Columbia.
- SIFT is invariance to image scale and rotation.



Feature Descriptor

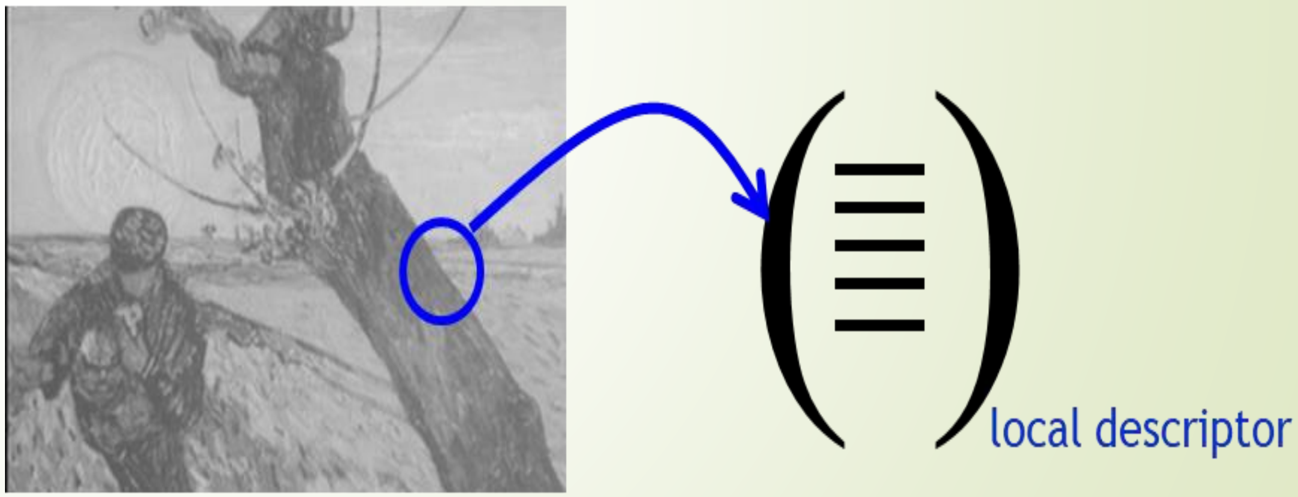
Advantages of SIFT

- Locality: features are local, so robust to occlusion and clutter (no prior segmentation)
- Distinctiveness: individual features can be matched to a large database of objects
- Quantity: many features can be generated for even small objects
- Efficiency: close to real-time performance
- Extensibility: can easily be extended to wide range of differing feature types, with each adding robustness

Feature Descriptor

SIFT Stage

- Scale-space extrema detection
- Keypoint localization
- **Orientation assignment**
- **Keypoint descriptor**



A 500x500 image gives about 2000 features

Feature Descriptor

SIFT Stage: Scale-space extrema detection

- *Identify locations and scales* that can be repeatably assigned under different views of the same scene or object.
- *Search for stable features across multiple scales* using a continuous function of scale.
- *The scale space of an image is a function $L(x,y,\sigma)$* that is produced from the convolution of a Gaussian kernel (at different scales) with the input image.
- Blurring

Convolution with a variable-scale Gaussian

$$L(x,y,\sigma) = G(x,y,\sigma) * I(x,y)$$

$$G(x,y,\sigma) = \frac{1}{2\pi\sigma^2} exp^{-(x^2+y^2)/\sigma^2}$$

G is the Gaussian Blur operator ,

I is an image and x,y are the location coordinates,

σ is the "scale" parameter

Feature Descriptor

SIFT Stage: **Scale-space extrema detection**

- DOG(Difference of Gaussian Kernel)
  - Use those blurred images to generate another set of images, the Difference of Gaussians (DoG).
  - These DoG images are great for finding out interesting keypoints in the image.
  - The difference of Gaussian is obtained as the difference of Gaussian blurring of an image with two different σ, let it be σ and kσ.

$$G(\boldsymbol{x},\boldsymbol{y},k\boldsymbol{\sigma}) - \boldsymbol{G}(\boldsymbol{x},\boldsymbol{y},\boldsymbol{\sigma})$$

$$D(x,y,\boldsymbol{\sigma}) = (G(x,y,k\boldsymbol{\sigma}) - G(x,y,\boldsymbol{\sigma}))*I(x,y)$$

$$= L(x,y,k\boldsymbol{\sigma}) - L(x,y,\boldsymbol{\sigma})$$

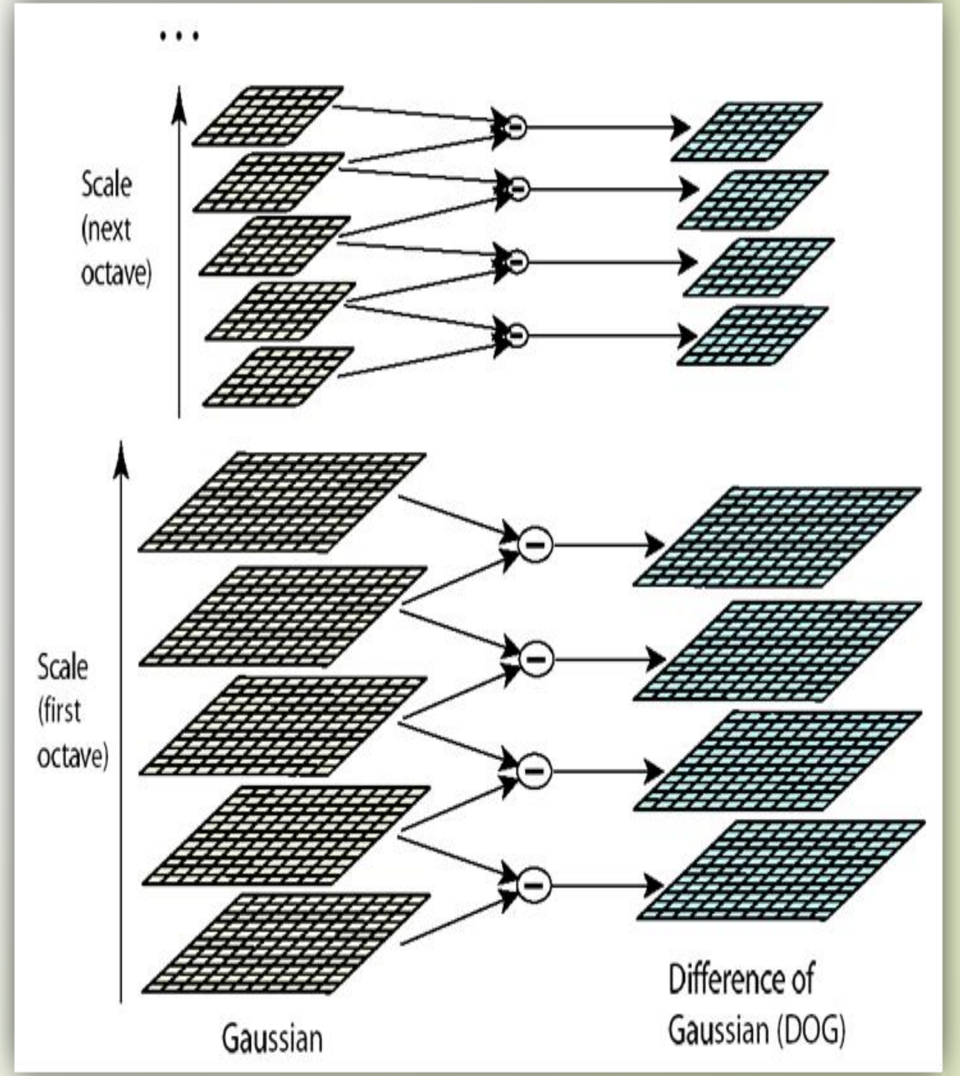
  - This process is done for different octaves of the image in the Gaussian Pyramid.

Feature Descriptor

SIFT Stage: Scale-space extrema detection

✯ DOG(Difference of Gaussian Kernel)

- Scale space is separated into octaves: Octave 1 uses scale σ, Octave 2 uses scale 2σ, etc.
- In each octave, the initial image is repeatedly convolved with Gaussians to produce a set of scale space images.
- Adjacent Gaussians are subtracted to produce the DOG.

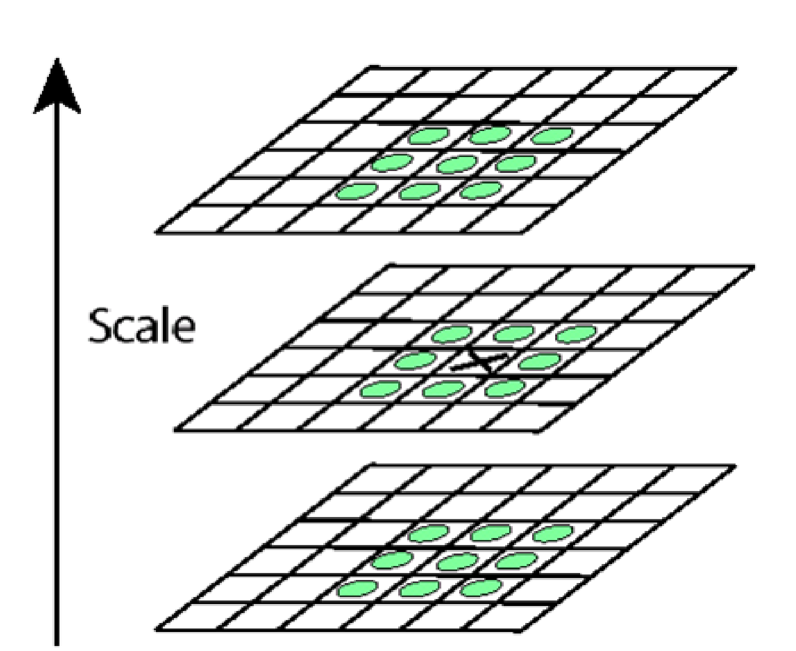


Feature Descriptor

SIFT Stage: **Scale-space extrema detection**

✯ Keypoint Finding

- Detect maxima and minima of difference-of-Gaussian in scale space
- Each point is compared to its 8 neighbors in the current image and 9 neighbors each in the scales above and below
- If it is a local extrema, it is a potential keypoint.



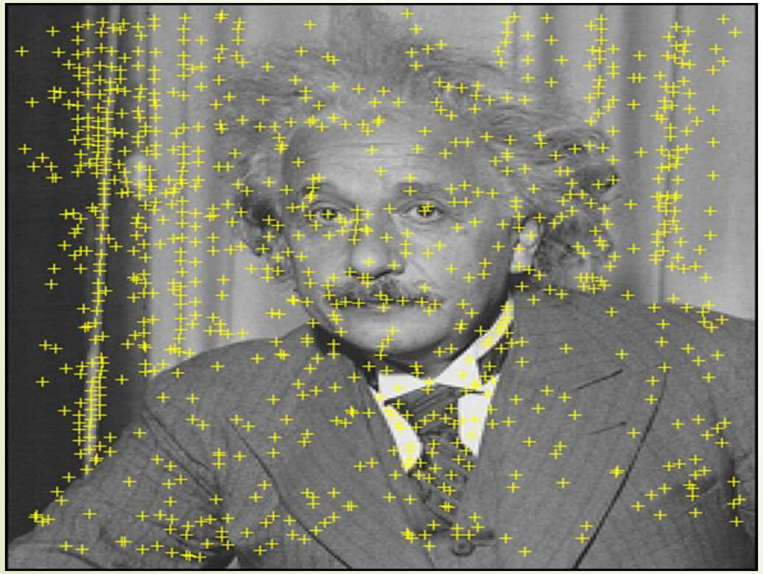
X is selected if it is larger or smaller than all 26 neighbors

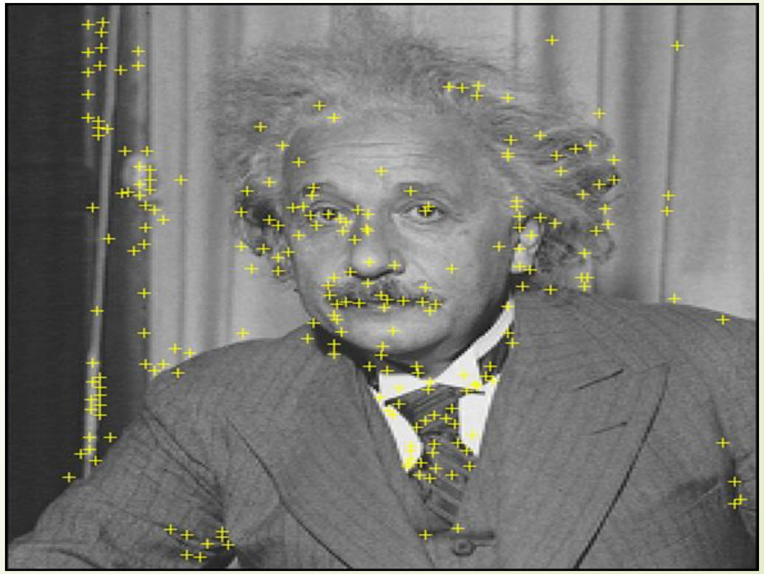
Feature Descriptor

42

SIFT Stage : Keypoint Lacalization

- There are still a lot of points, some of them are not good enough.
- The locations of keypoints may be not accurate.
- Some of them lie along an edge, or they don’t have enough contrast.
- If the intensity at this extrema is less than threshold value, then it is rejected.
- Edges are removed using the eigen values and their ratios.
- Key points with low contrast or poorly localized along edges are discarded to improve robustness.

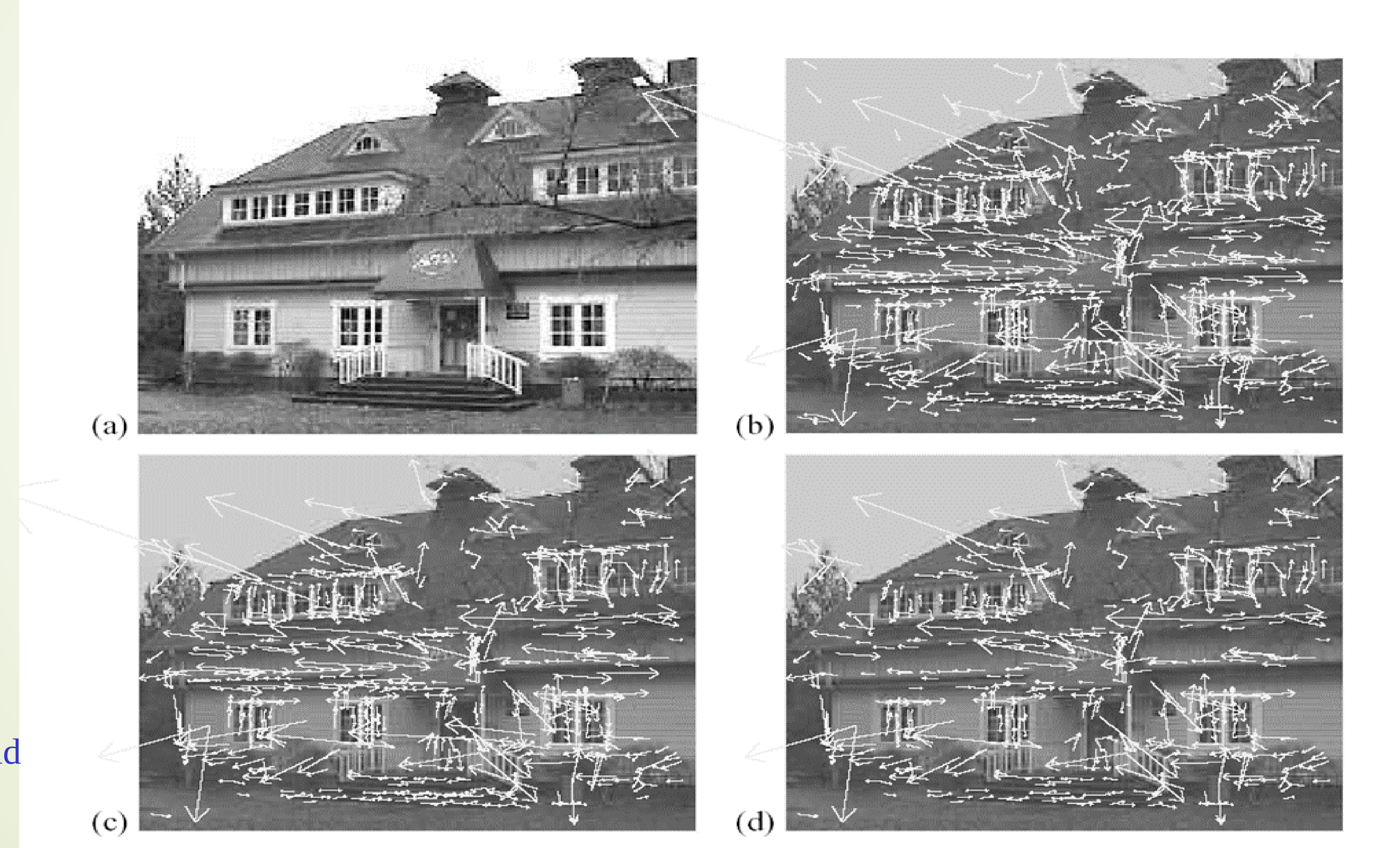




Feature Descriptor

SIFT Stage : Keypoint Lacalization

233x189



832

initial keypoints

729

keypoints after gradient threshold

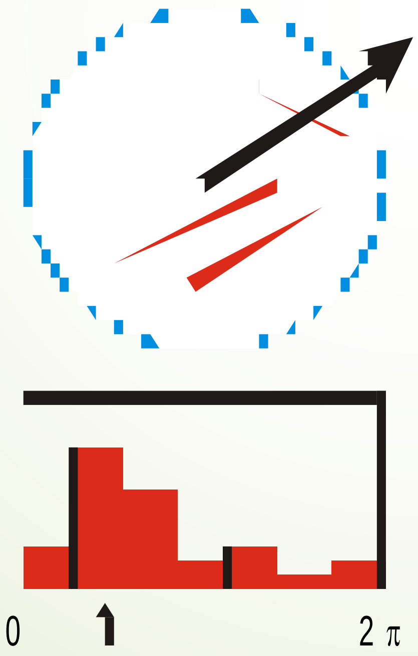
536

keypoints after ratio threshold

Feature Descriptor

SIFT Stage : Orientation Assignment

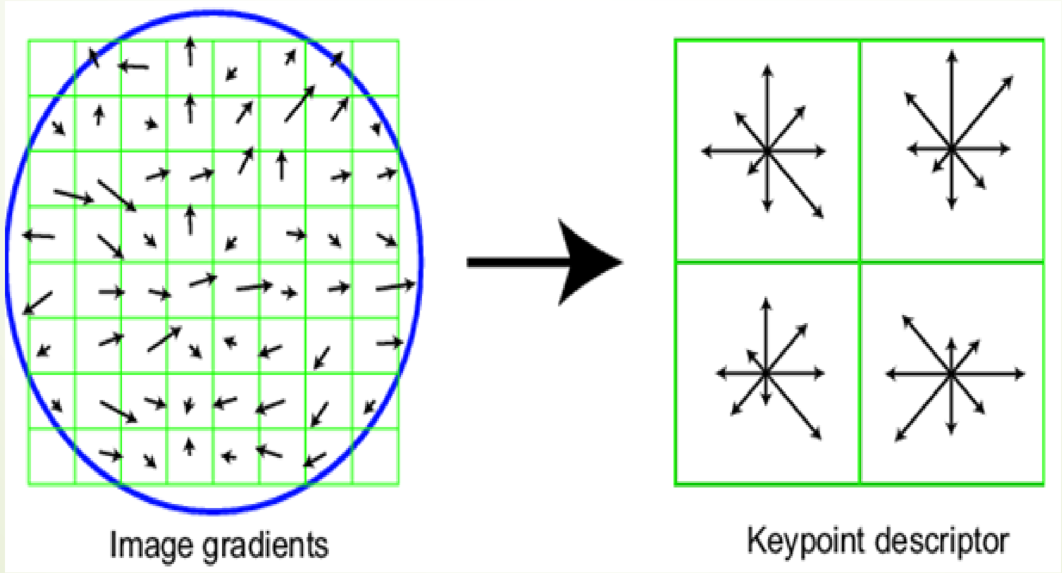
- The next thing is to assign an orientation to each keypoint to make it rotation invariance.
- An orientation histogram with 36 bins covering 360 degrees is created.
- The highest peak in the histogram is taken and any peak above 80% of it is also considered to calculate the orientation.
- It creates keypoints with same location and scale, but different directions.



Feature Descriptor

SIFT Stage : Keypoint Descriptor

- At this point, each keypoint has
  - location
  - Scale (the radius of the region)
  - Orientation (an angle expressed in radians)
- Next is to compute a descriptor for the local image region about each keypoint.



Based on 16*16 patches
4*4 subregions
8 bins in each subregion
4*4*8=128 dimensions in total

In experiments, 4x4 arrays of 8 bin histogram is used, a total of 128 features for one keypoint

Feature Descriptor

PCA-SIFT

- The dimensionality of SIFT is very high, i.e., 128 D for each keypoint
- Reduce the dimensionality using linear dimensionality reduction
- In this case, principal component analysis (PCA)
- A more compact descriptor
- More robust, faster

Feature Descriptor

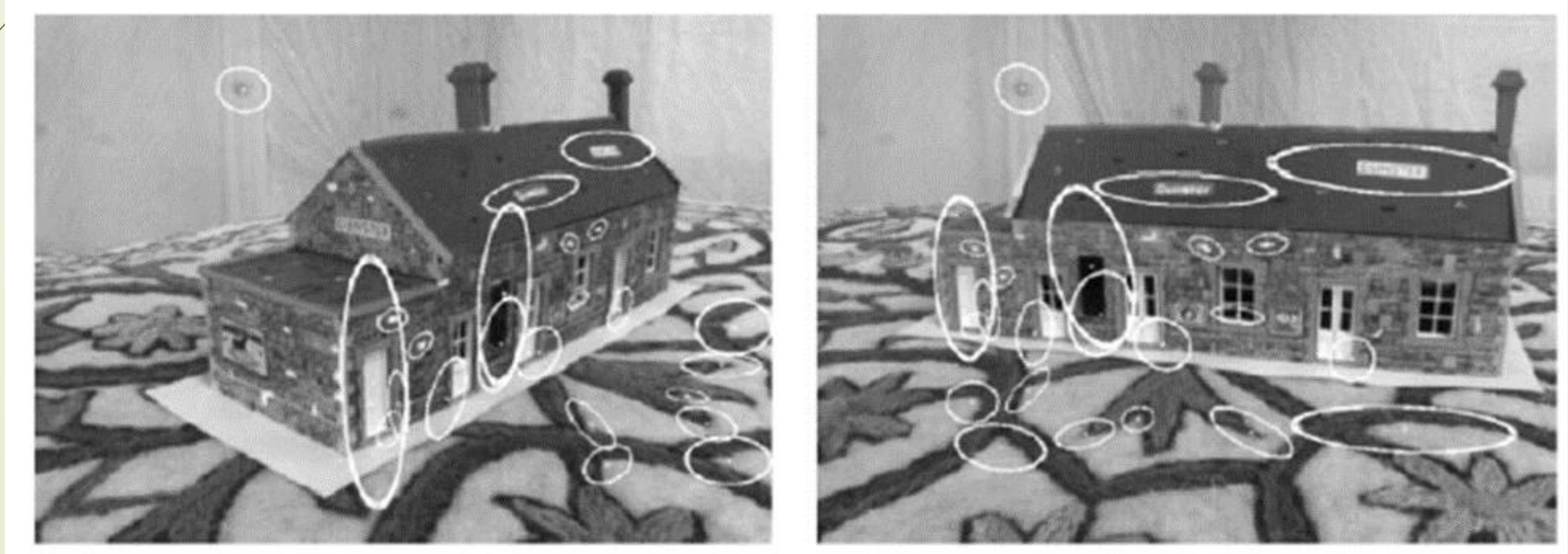
Gradient Location-Orientation Histogram (GLOH)

- It is a robust image descriptor that can be used in computer vision tasks.
- It is a SIFT-like descriptor that considers more spatial regions for the histograms.
- An intermediate vector is computed from 17 location and 16 orientation bins, for a total of 272-dimensions.
- Principal components analysis (PCA) is then used to reduce the vector size to 128

Feature Descriptor

Affine Invariance

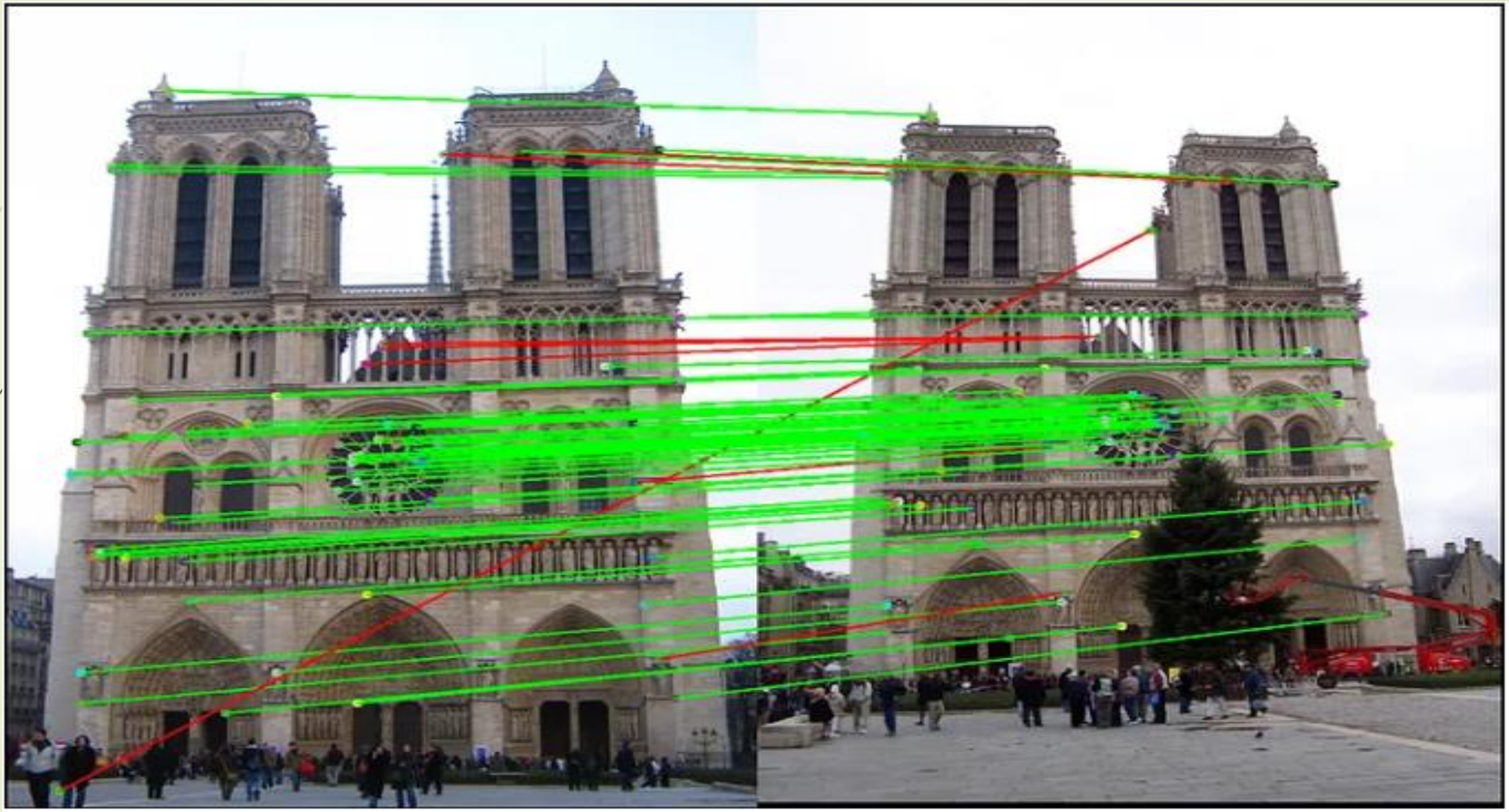
- Affine-invariant detectors not only respond at consistent locations after scale and orientation changes,
- they also respond consistently across affine deformations such as (local) perspective foreshortening.



Feature Matching and Tracking

- It is the task of establishing correspondences between two images of the same scene/object.
- Once the features and their descriptors have been extracted from two or more images, the next step is to establish some preliminary feature matches between these images.
- Generally, the ***performance*** of matching methods based on interest points and the choice of associated image descriptors.
- Algorithms
  - Brute-Force Matcher
  - FLANN (Fast Library for Approximate Nearest Neighbors)

Feature Matching and Tracking



Feature Matching and Tracking

Feature Matching

- Features matching or generally image matching,
  - is the task of establishing correspondences between two images of the same scene/object.
  - such as image registration, image retrieval, object recognition, etc.
- Firstly, detecting a set of interest points each associated with image descriptors from image data.
- The next step is to establish some preliminary feature matches between these images.

52

Feature Matching and Matching

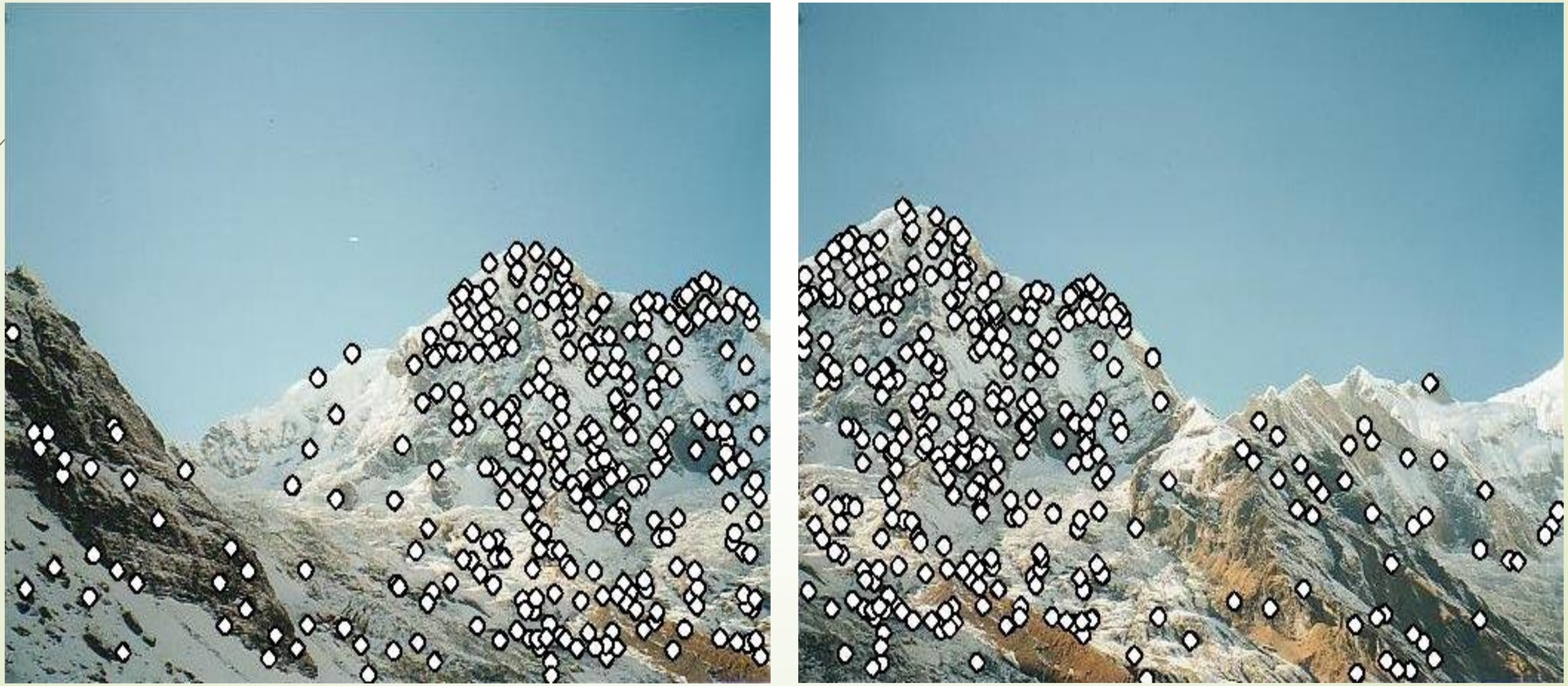
How do we build panorama?

- Need to match (align) images

Feature Matching and Tracking

Matching with Features

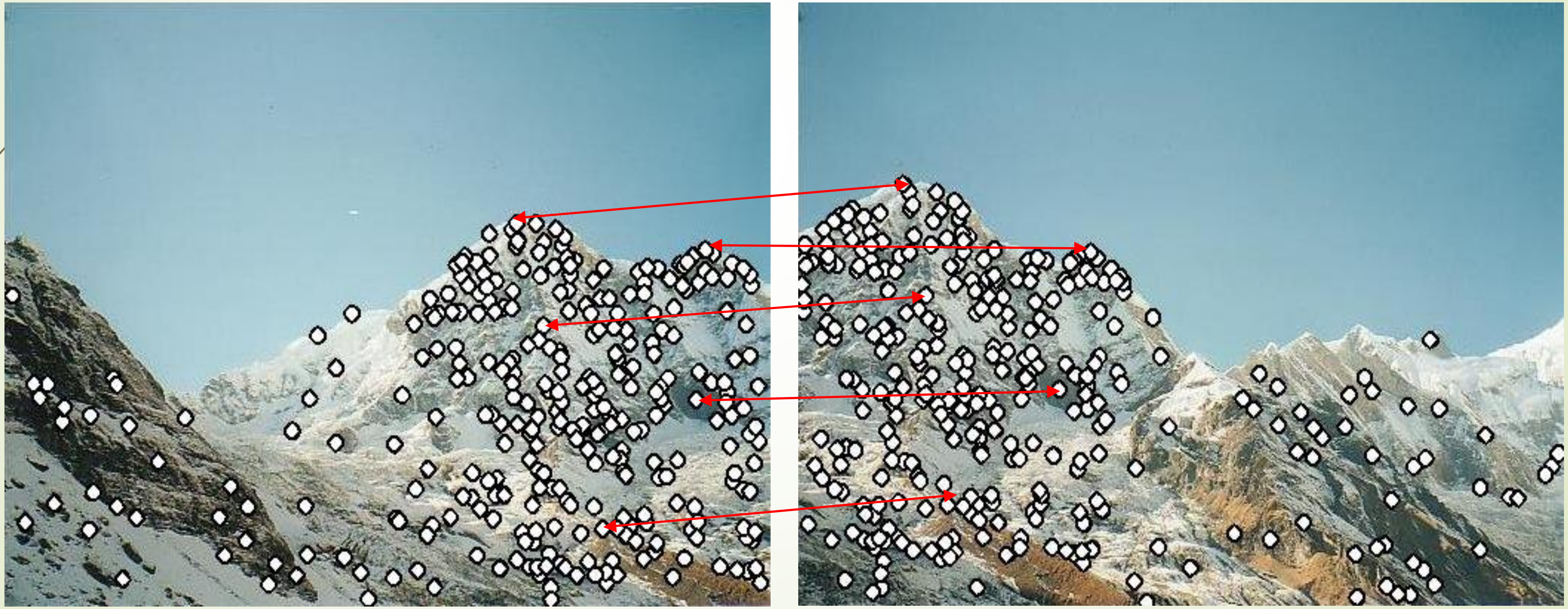
- Detect feature points in both images



Feature Matching and Tracking

Matching with Features

- Detect feature points in both images
- Find corresponding pairs



Feature Matching and Tracking

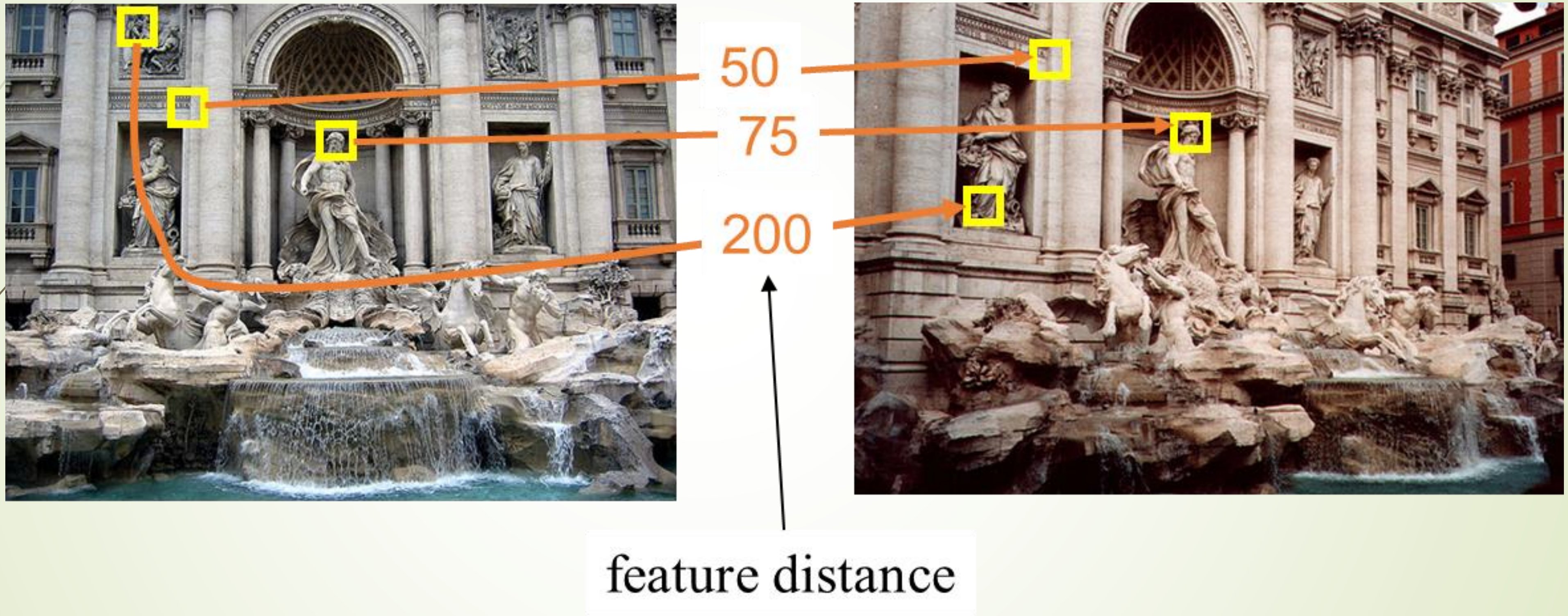
Matching with Features

- Detect feature points in both images
- Find corresponding pairs
- Use these pairs to align images

Feature Matching and Tracking

Distance Threshold

- How can we measure the performance of a feature matcher?



The distance threshold affects performance

True positives = # of detected matches that are correct

Suppose we want to maximize these—how to choose threshold?

False positives = # of detected matches that are incorrect

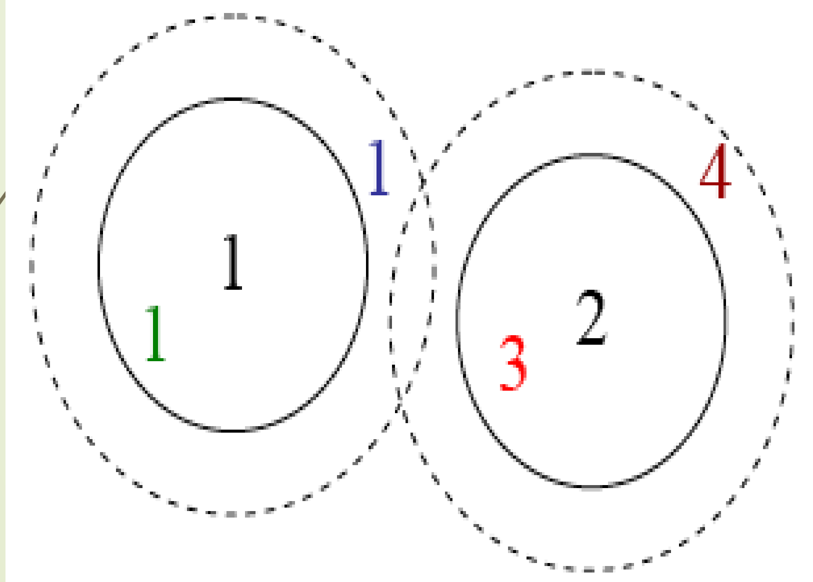
Suppose we want to minimize these—how to choose threshold?

Feature Matching and Tracking

Which threshold to use?

- Setting the high threshold -too many false positives
- Setting the too low threshold - many false negatives

Black 1, 2: features



At the current threshold setting (the solid circles),

- –Green 1: true positive
- –Blue 1: false negative
- – 3: false positive
- –4: true negative

If we set the threshold higher (the dashed circles),

- – Green 1: true positive
- – Blue 1: true positive
- – 3: false positive
- – 4: false positive

Feature Matching and Tracking

Evaluating the Results

- TP: true positives, i.e., number of correct matches
- FN: false negatives, matches that were not correctly detected
- FP: false positives, proposed matches that are incorrect
- TN: true negatives, non-matches that were correctly rejected.

True positive rate (**Recall**), TPR = TP/(TP+FN) = TP/P

False positive rate, FPR = FP/(FP+TN)

Positive predictive Value (**Precision**) = TP/(TP+FP)

Accuracy (ACC) = (TP+TN)/(P+N)

Feature Matching and Tracking

Confusion Matrix

	True Match	True Non-match	
Predicted Match	TP=18	FP=4	P'=22
Predicted Non-match	FN=2	TN=76	N'=78
	P=20	N=80	Total =100

PPV=0.82

ACC=0.94

TPR=0.90	FPR=0.05

Ideally, TPR = 1, FPR =0

Feature Matching and Tracking

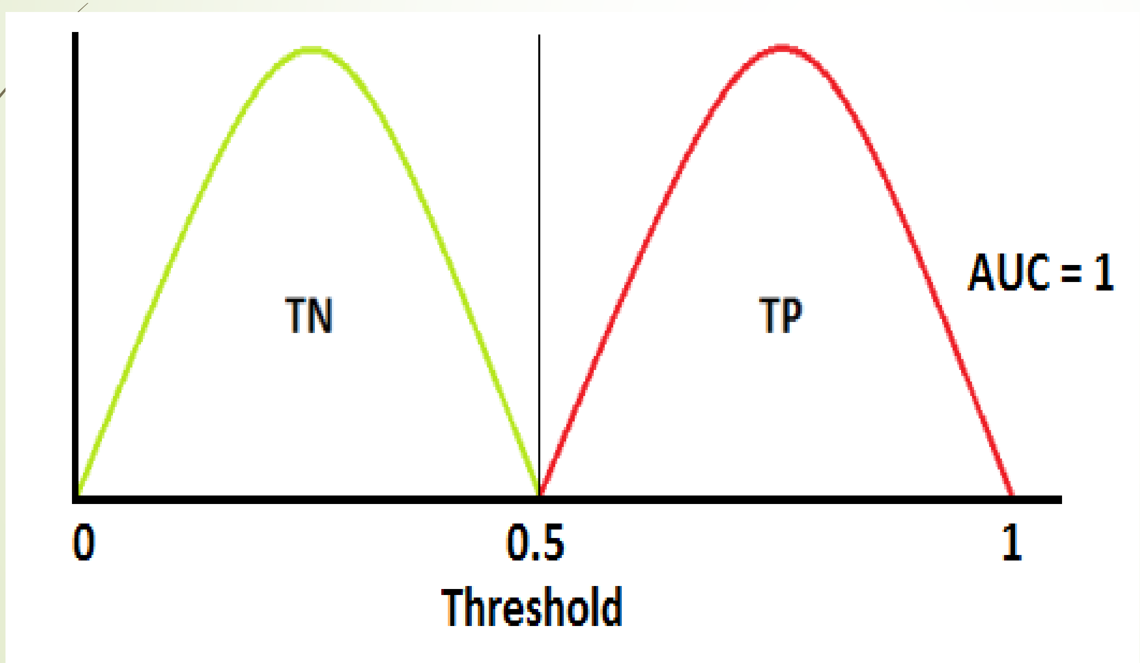
Measuring Performance

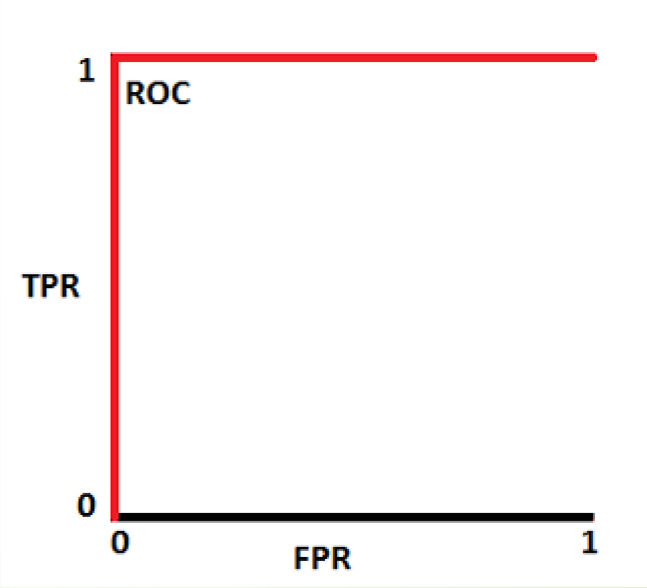
- To evaluate the performance of the classification models, we use AUC (Area Under The Curve) and ROC (Receiver Operating Characteristics) curve.
- **ROC** is a probability curve,
  - It is plotted with TPR against the FPR where TPR is on y-axis and FPR is on the x-axis.
- AUC represents degree or measure of separability and higher the AUC, better the model is at predicting 0s as 0s and 1s as 1s.
  - An excellent model has AUC near to the 1.
  - A poor model has AUC near to the 0.

Feature Matching and Tracking

Measuring Performance

- When two curves don’t overlap at all means model has an ideal measure of separability.
- It is perfectly able to distinguish between positive class and negative class.

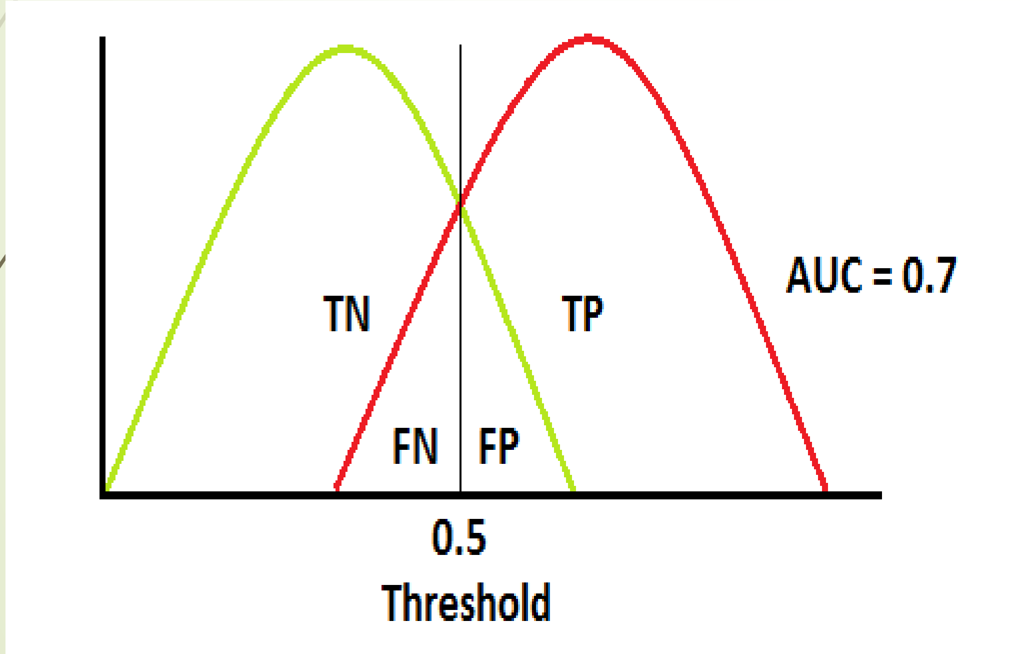


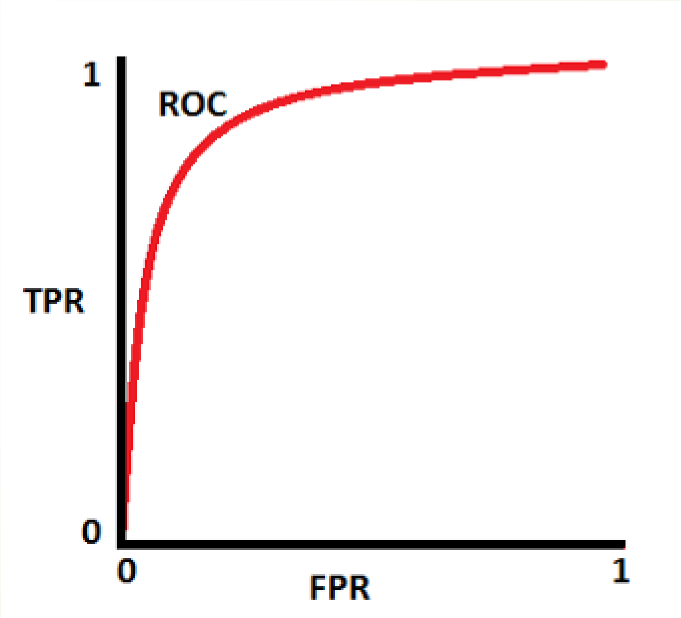


Feature Matching and Tracking

Measuring Performance

- **AUC is 0.7**, it means there is 70% chance that model will be able to distinguish between positive class and negative class.

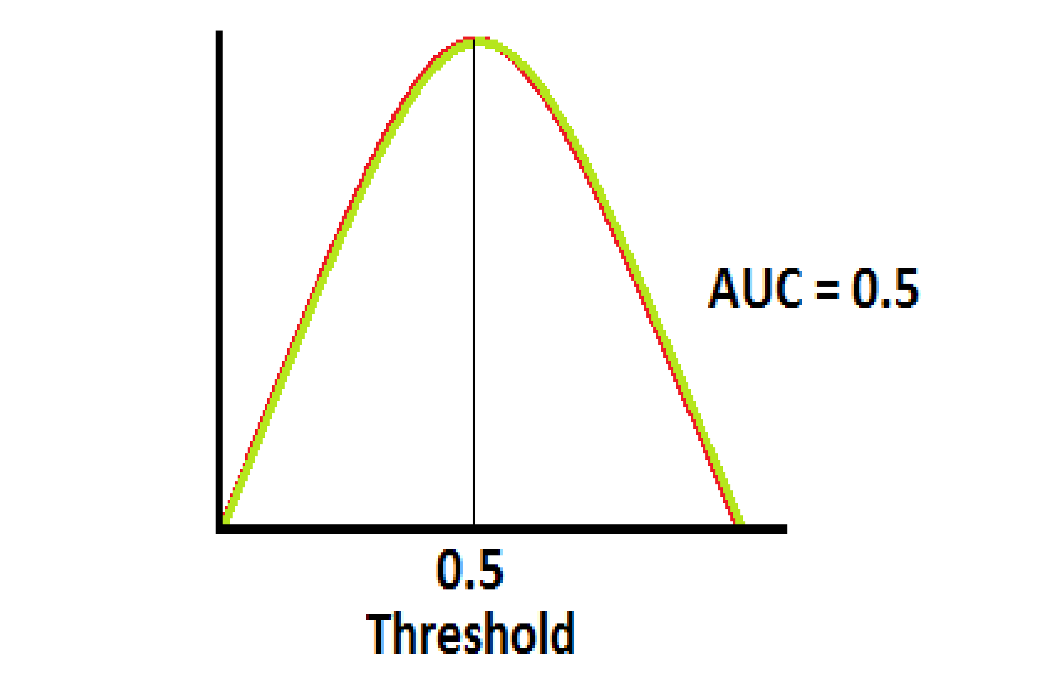


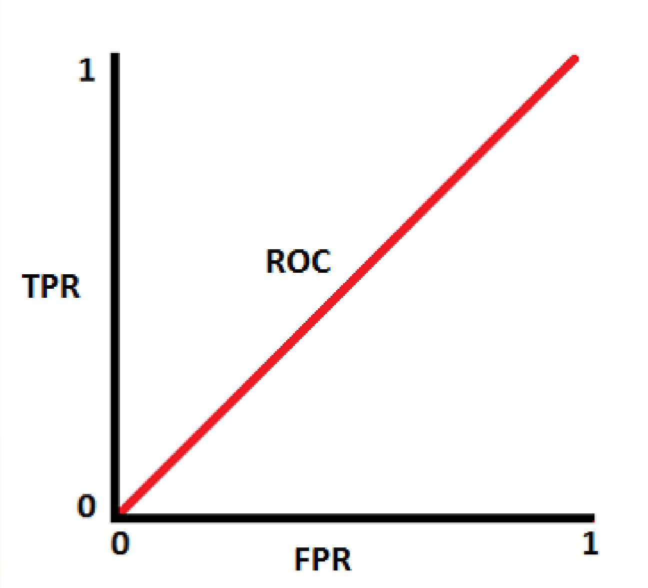


Feature Matching and Tracking

Measuring Performance

- When AUC is approximately 0.5, model has no discrimination capacity to distinguish between positive class and negative class.

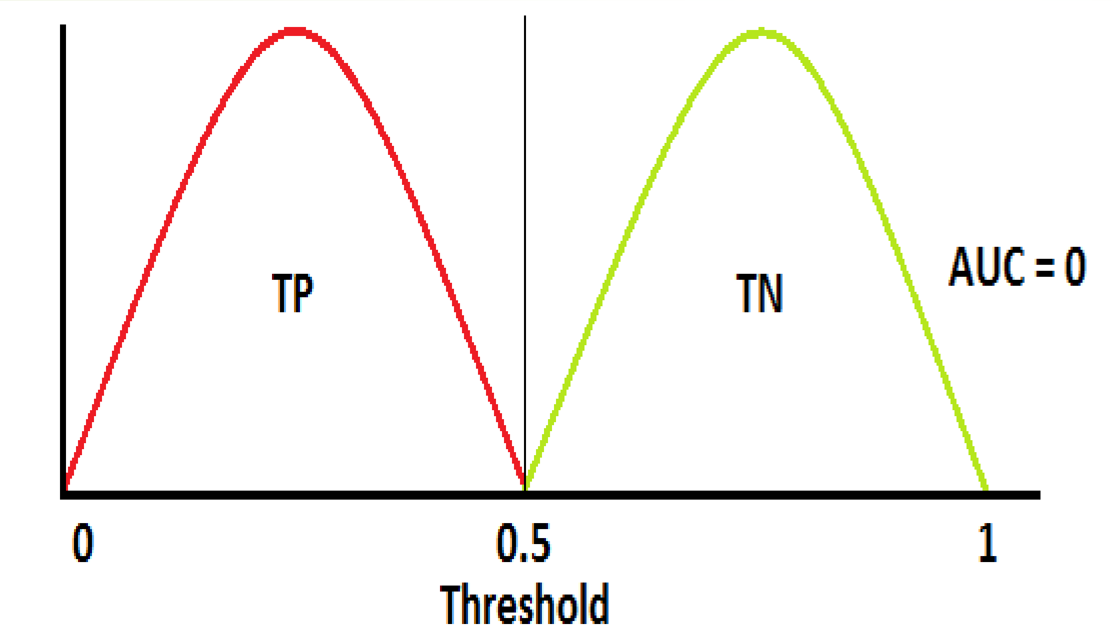


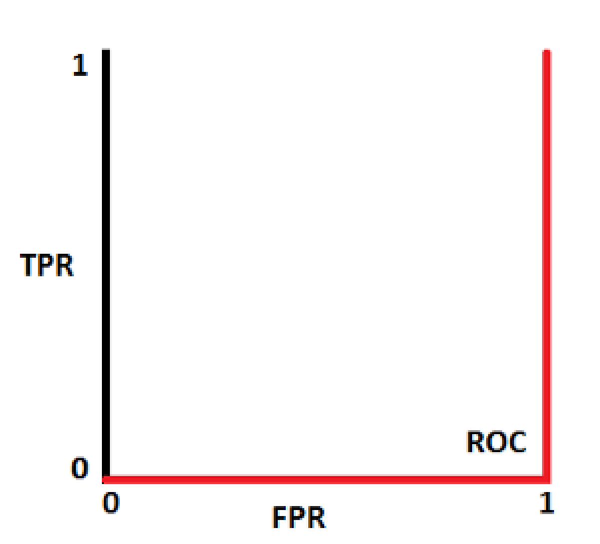


Feature Matching and Tracking

Measuring Performance

- When AUC is approximately 0, model is actually reciprocating the classes.
- It means, model is predicting negative class as a positive class and vice versa.





Feature Matching and Tracking

Feature Tracking

- Find features in all candidate images
  - Detect in one then search in others
  - Detect and track : for video tracking applications
- Subsequent frames
  - Sum Square Difference (SSD) works well
  - Normalized Cross-Correlation (NCC) is better if there is brightness change
  - Hierarchical search strategy when search range is large: matches in lower resolution images to provide better initial guesses and speed up the search
- Long image sequence : large appearance change
  - Whether to continue matching against the originally detected patch

Fail when original patch apperance changes such as foreshortening.

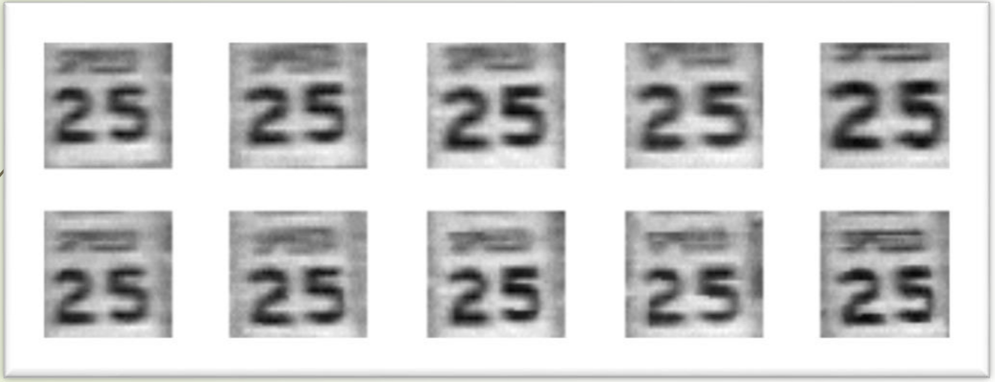
- (OR) Re-sample each subsequent frame at the matching location.

Risk: features drift from its original location

Feature Matching and Tracking

Affine Motion Model

- Compare patches in neighboring frames using a translational model.
- Use the location to initialize an affine registration between the patch in the current frame and the base frame.



Kanade–Lucas–Tomasi (KLT) tracker

Detect new features in regions where the tracking has fail

- The area around the predicted feature location is searched with an incremental registration algorithm.

Feature Matching and Tracking

Affine Motion Model

- Example

Real-time head tracking using the fast trained classifiers of Lepetit, Pilet, and Fua (2004) 2004 IEEE.

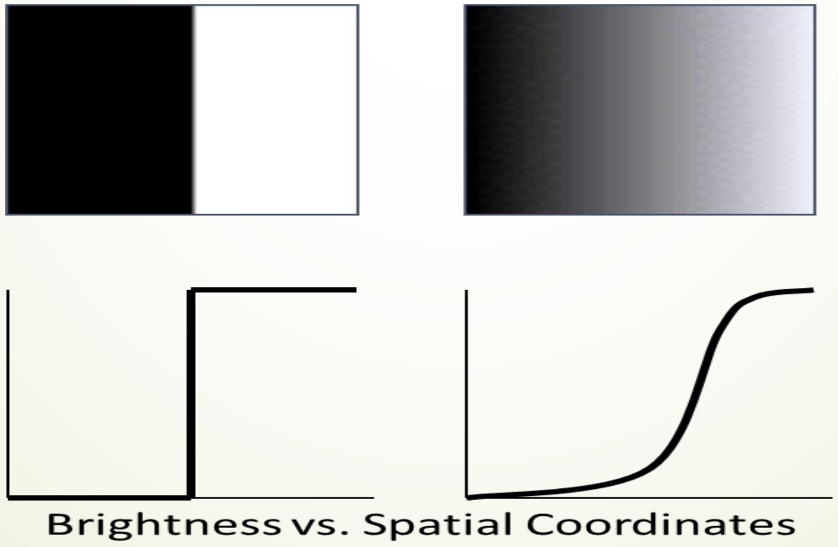
Feature Detection and Matching

Algorithm for Feature Detection and Matching

- Find a set of distinctive keypoints
- Define a region around each keypoint
- Extract and normalize the region content
- Compute a local descriptor from the normalized region
- Match local descriptors

Edges

- Edges can be defined as the points in an image where the intensity of pixels changes sharply.
- These changes often correspond to the physical boundaries of objects within the scene.
- Edges are those places in an image that correspond to object boundaries.
- Edges are pixels where image brightness changes abruptly.



Edges

Characteristics

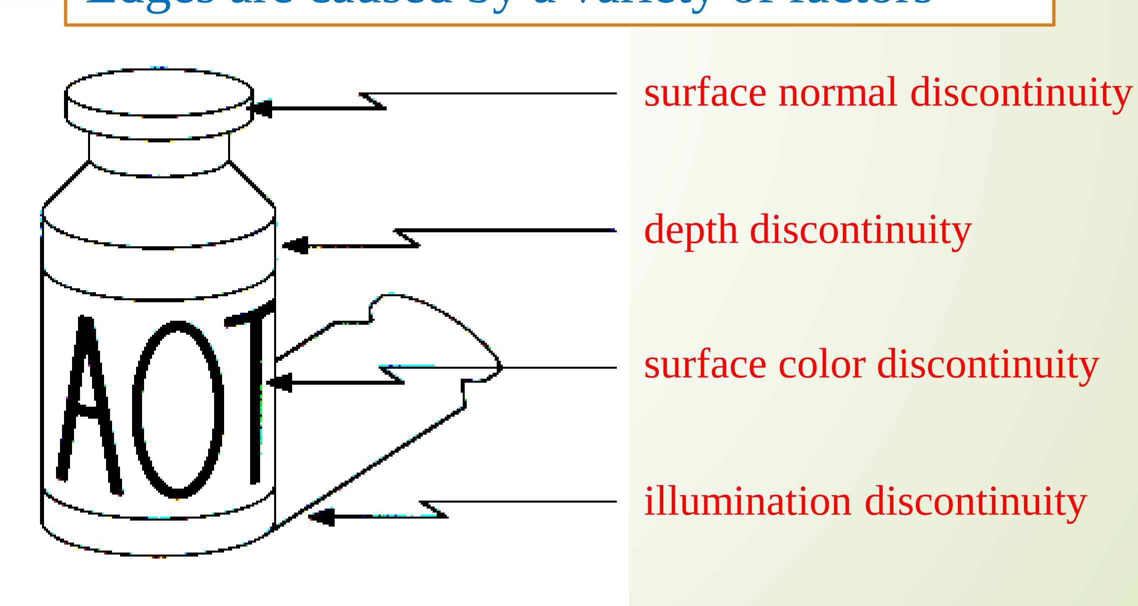
- Gradient Magnitude
  - The edge strength is determined by the gradient magnitude, which measures the *rate of change in intensity.*
- Gradient Direction
  - The direction of the edge is perpendicular to the direction of the gradient, indicating *the orientation of the boundary.*
- Localization
  - Edges should be well-localized, meaning they should accurately correspond to *the true boundaries in the image.*
- Noise Sensitivity
  - Edges can be affected by noise, making it essential to use techniques that can distinguish between actual edges and noise.

Edges

Origin of Edges

- Geometric events
  - surface orientation (boundary) discontinuities
  - color and texture discontinuities
  - depth discontinuities
- Non-geometric events
  - illumination changes
  - specularities
  - shadows
  - inter-reflections

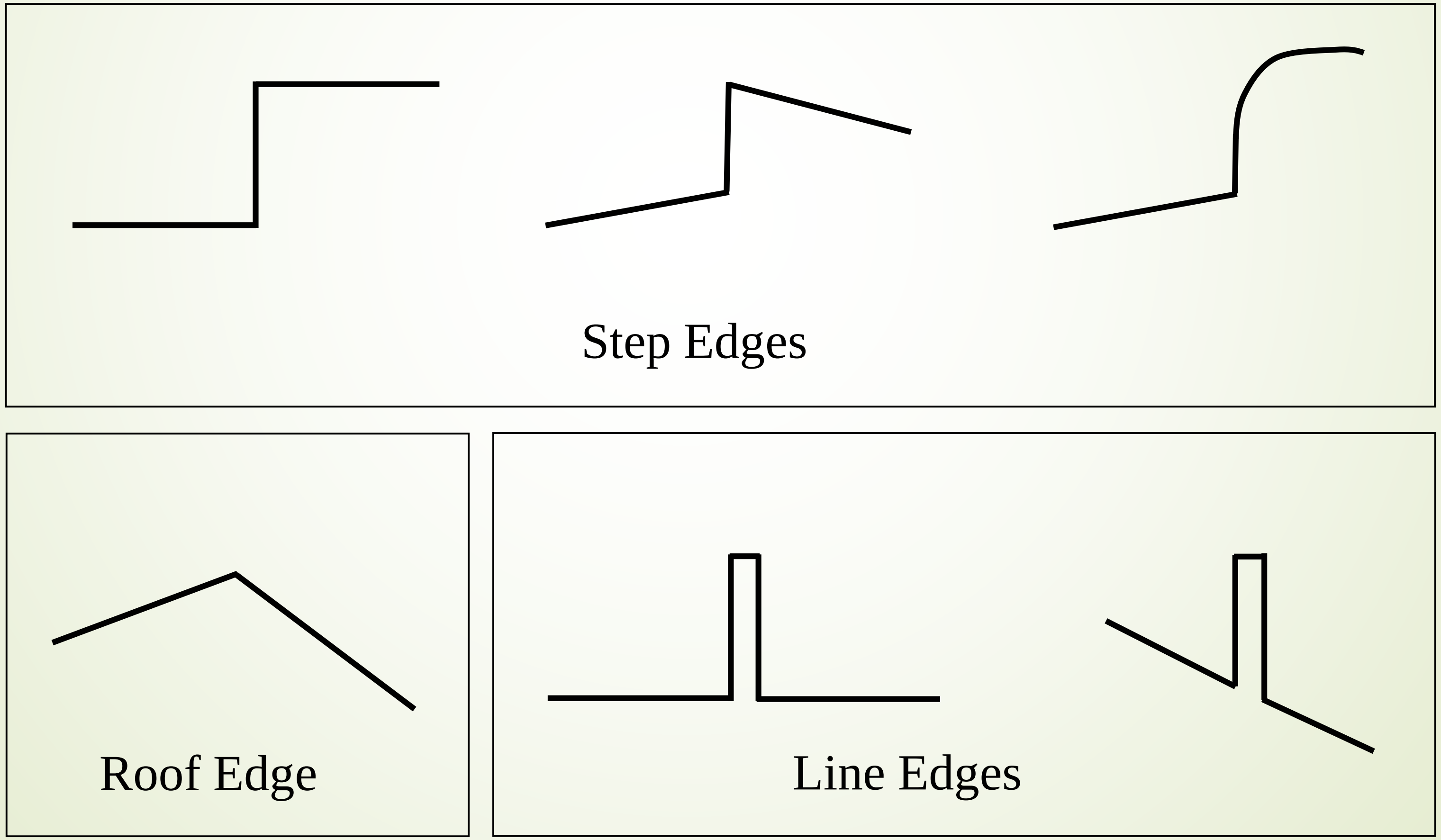
Edges are caused by a variety of factors



Edges

Types of Edge

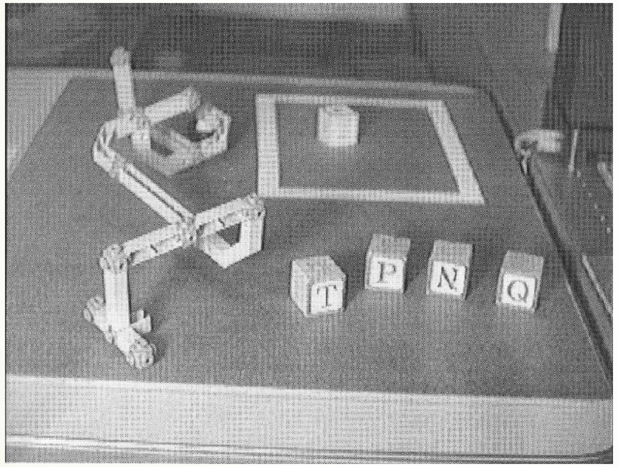
- Edges can be classified into several types based on their appearance and the way intensity changes occur:

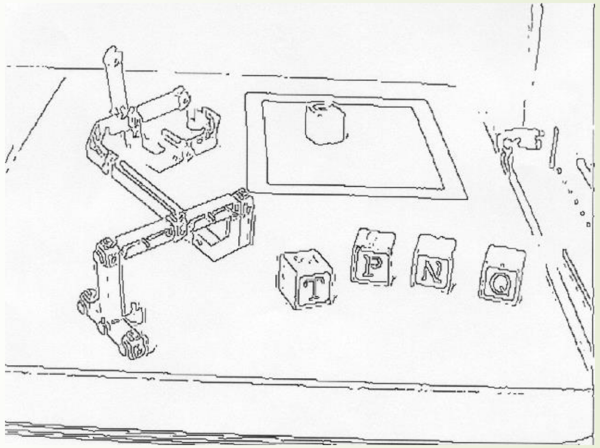


Edges

Why is edge detection useful?

- Important features can be extracted from the edges of an image (e.g., corners, lines, curves).
- These features are used by higher-level computer vision algorithms (e.g., recognition).





To Detect Edges

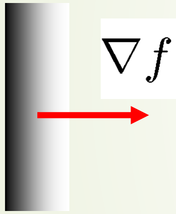
- Edge information in an image is found by looking at the relationship a pixel has with its neighborhoods.
- If a pixel's gray-level value is similar to those around it, there is probably not an edge at that point.
- If a pixel's has neighbors with widely varying gray levels, it may present an edge point.

Edges

Image Gradient

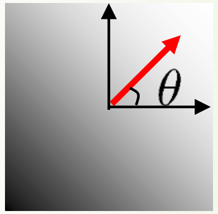
$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

- The gradient points in the direction of most rapid change in intensity.



$$\nabla f = \left[\frac{\partial f}{\partial x}, 0\right]$$

$$\nabla f = \left[0, \frac{\partial f}{\partial y}\right]$$



$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$$

- The gradient direction is given by:

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

- how does this relate to the direction of the edge?
- The *edge strength* is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Edges

Main Steps in Edge Detection

- Smoothing:

  Suppress as much noise as possible, without destroying true edges.

- Enhancement:

  Apply differentiation to enhance the quality of edges (i.e., sharpening).

- Thresholding:

  Determine which edge pixels should be discarded as noise and which should be retained (i.e., threshold edge magnitude).

- Localization:

  Determine the exact edge location.

Edges

Criteria for Optimal Edge Detection

- Good detection
  - Minimize the probability of false positives (i.e., spurious edges).
  - Minimize the probability of false negatives (i.e., missing real edges).
- Good localization
  - Detected edges must be as close as possible to the true edges.
- Single response
  - Minimize the number of local maxima around the true edge.

Edges

Methods of Edge Detection

- First Order Derivative / Gradient-Based Methods
  - Roberts Operator
  - Sobel Operator
  - Prewitt Operator
- Second Order Derivative
  - Laplacian
  - Laplacian of Gaussian (LoG)
  - Difference of Gaussian (DoG)
- Optimal Edge Detection
  - Canny Edge Detection
- Multi-Scale Edge Detection
  - Wavelet Transform
- Machine Learning and Deep Learning-Based Methods
  - Structured Forests
  - Convolutional Neural Networks (CNNs)

Edges

Roberts Operator

- It is one of the oldest and simplest edge detection algorithms to implement.
- It uses two 2x2 matrices to find the changes in the x and y directions.
  - Mark edge point only
  - No information about edge orientation
  - Work best with binary images
- To perform edge detection with the Roberts operator, convolve the original image with the following two kernels:

$$h_1 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \qquad h_2 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

- The gradient is :
  - original image
  - an image formed by convolving with the first kernel
  - an image formed by convolving with the second kernel
- The direction of the edge's gradient is $\alpha = \operatorname{atan}\left(\frac{G_y}{G_x}\right)$

Edges

Sobel Operator

- It is also a derivate mask and is used for edge detection.
- Sobel operator is also used to detect two kinds of edges in an image:
  - Vertical direction
  - Horizontal direction
- The masks are as follows:

$$x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \qquad y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

- Edge Magnitude $= \sqrt{x^2 + y^2}$
- Edge Direction $= \tan^{-1}\left[\frac{y}{x}\right]$

Edges

Prewitt Operator

- Similar to the Sobel, with different mask coefficients:
- The masks are as follows:

$$x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \qquad y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

- It detects two types of edges
  - Horizontal edges
  - Vertical Edges

- Edge Magnitude $= \sqrt{x^2 + y^2}$
- Edge Direction $= \tan^{-1}\left[\frac{y}{x}\right]$

Edges

Laplacian Operator

- Laplacian is a second order derivative mask.
- It can be further divided into positive laplacian and negative laplacian.
- Edge magnitude is approximated in digital images by a convolution sum.
- The sign of the result (+ or -) from two adjacent pixels provide edge orientation.
- **Positive Laplacian Operator**
  - Center element should be negative
  - Corner elements is zero and
  - Rest of all the elements in the mask should be 1.

0	**1**	**0**
1	**-4**	**1**
0	**1**	**0**

- **Negative Laplacian Operator**
  - Center element should be positive
  - Corner elements should be zero and
  - Rest of all the elements in the mask should be -1.

0	**-1**	**0**
-1	**4**	**-1**
0	**-1**	**0**

Edges

Canny Edge Detection

- Canny edge detection is a technique to extract useful structural information from different vision objects.
- Dramatically reduce the amount of data to be processed.
- It has been widely applied in various computer vision systems.
- It was developed by John F. Canny in 1986.
- Canny also produced a computational theory of edge detection explaining why the technique works.

Edges

Steps of Canny Edge Detection Algorithm

- The Canny edge detection algorithm is composed of 5 steps:
  1. Noise reduction
  2. Gradient calculation
  3. Non-maximum suppression
  4. Double threshold
  5. Edge Tracking by Hysteresis

Edges

Canny Edge Detection

- Steps 1: Noise Reduction
- Edge detection results are highly sensitive to image noise.
- One way to get rid of the noise on the image, is by applying Gaussian blur to smooth it.
- Perform a Gaussian blur on the image.
- The blur removes some of the noise before further processing the image.
- To do so, image convolution technique is applied with a Gaussian Kernel (3x3, 5x5, 7x7 etc…).

Original image (left) — Blurred image with a Gaussian filter (sigma=1.4 and kernel size of 5x5)

Edges

Canny Edge Detection

- Steps 2: Gradient Calculation
- The Gradient calculation step detects the edge intensity and direction by calculating the gradient of the image using edge detection operators.
- The edge detection operator (such as Roberts, Prewitt, or Sobel) returns a value for the first derivative in the horizontal direction (Gx) and the vertical direction (Gy).
- From this the edge gradient and direction can be determined:

$$\mathbf{G} = \sqrt{{\mathbf{G}_x}^2 + {\mathbf{G}_y}^2} \qquad \angle G = \arctan(G_y/G_x)$$

Blurred image (left) — Gradient intensity (right)

Edges

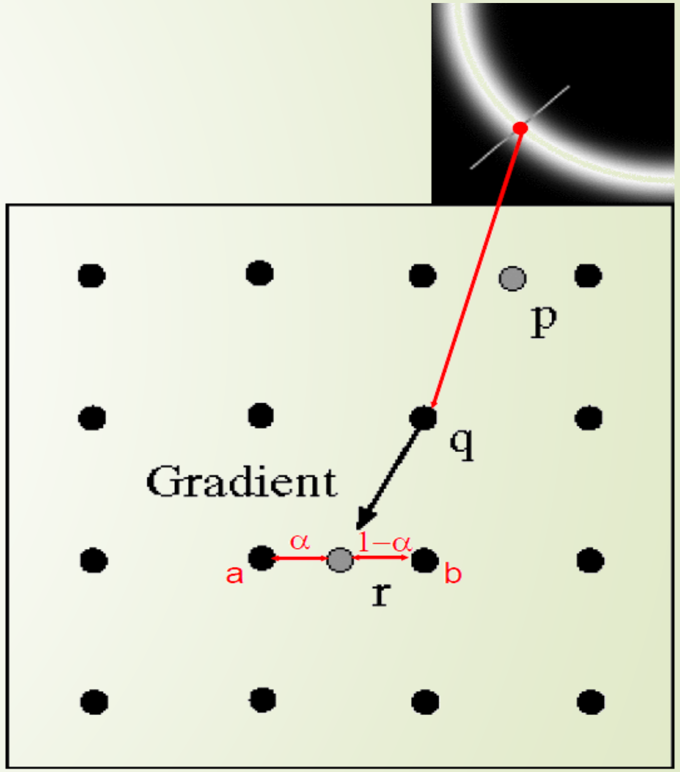
Canny Edge Detection

- Steps 3: Non Maximum Supression
- Ideally, the final image should have thin edges. Thus, we must perform non-maximum suppression to thin out the edges.
- Applied to find the locations with the sharpest change of intensity value.
- The algorithm for each pixel in the gradient image is:
  - Compare the edge strength of the current pixel with the edge strength of the pixel in the positive and negative gradient directions.
  - If the edge strength of the current pixel is the largest compared to the other pixels in the mask with the same, the value will be preserved. Otherwise, the value will be suppressed.

Edges

Canny Edge Detection

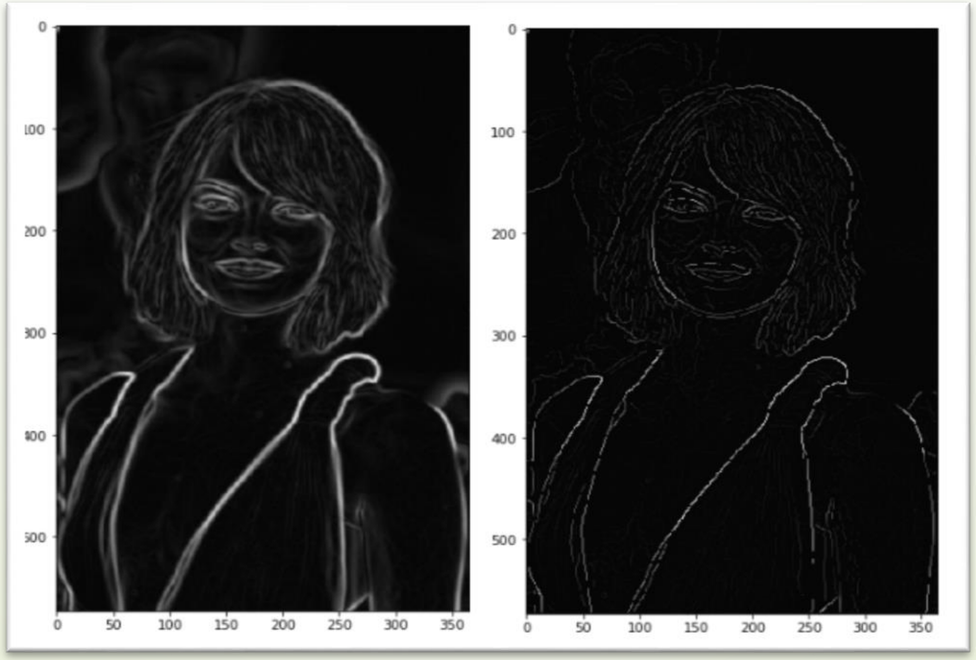
- Steps 3: Non Maximum Supression
- Non maximum suppression works by finding the pixel with the maximum value in an edge.
- In this image, it occurs when pixel q has an intensity that is larger than both p and r.
- where pixels p and r are the pixels in the gradient direction of q.
- If this condition is true, then we keep the pixel, otherwise we set the pixel to zero (make it a black pixel)



Edges

Canny Edge Detection

- Steps 3: Non Maximum Supression
- After application of non-maximum suppression, remaining edge pixels provide a more accurate representation of real edges in an image.
- However, some edge pixels remain that are caused by noise and color variation (some pixels seem to be brighter than others).



Edges

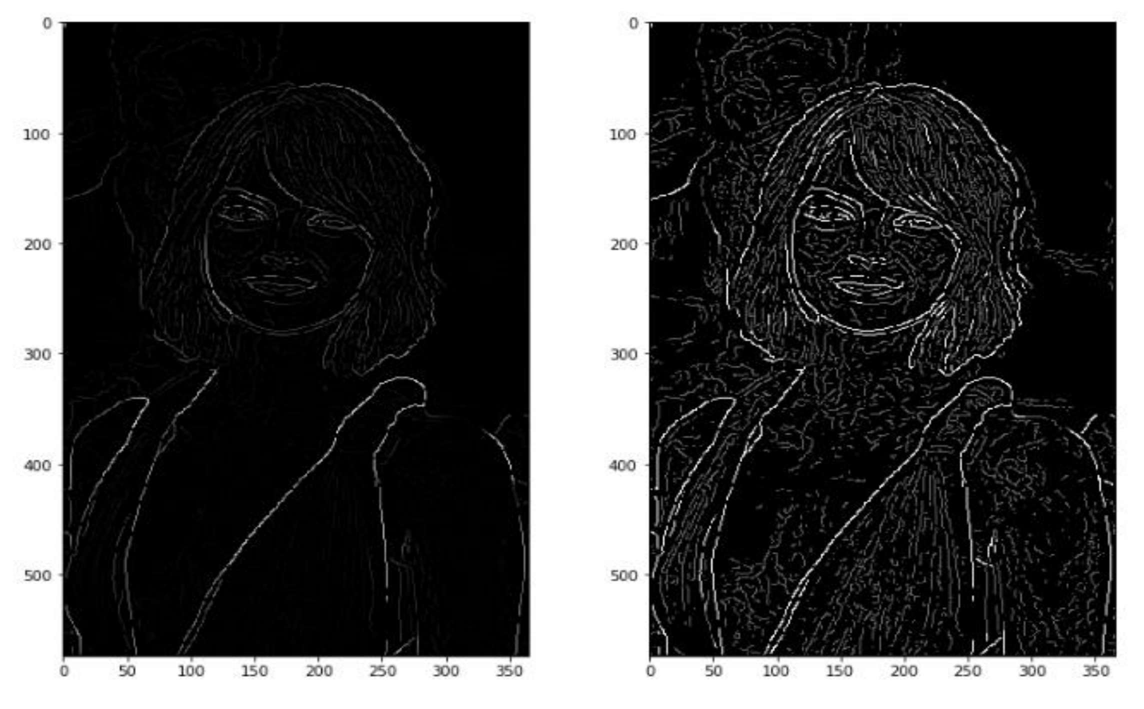
Canny Edge Detection

- **Steps 4: Double Threshold**
- The double threshold step aims at identifying 3 kinds of pixels: *strong, weak, and non-relevant*:
- *Strong pixels* are pixels that have an intensity so high that we are sure they contribute to the final edge.
- *Weak pixels* are pixels that have an intensity value that is not enough to be considered as strong ones, but yet not small enough to be considered as non-relevant for the edge detection.
- Other pixels are considered as *non-relevant* for the edge.

Edges

Canny Edge Detection

- Steps 4: Double Threshold
- *High threshold* is used to identify the ***strong pixels*** (intensity higher than the high threshold)
- *Low threshold* is used to identify the ***non-relevant pixels*** (intensity lower than the low threshold)
- All pixels having intensity *between both thresholds* are flagged as ***weak***.

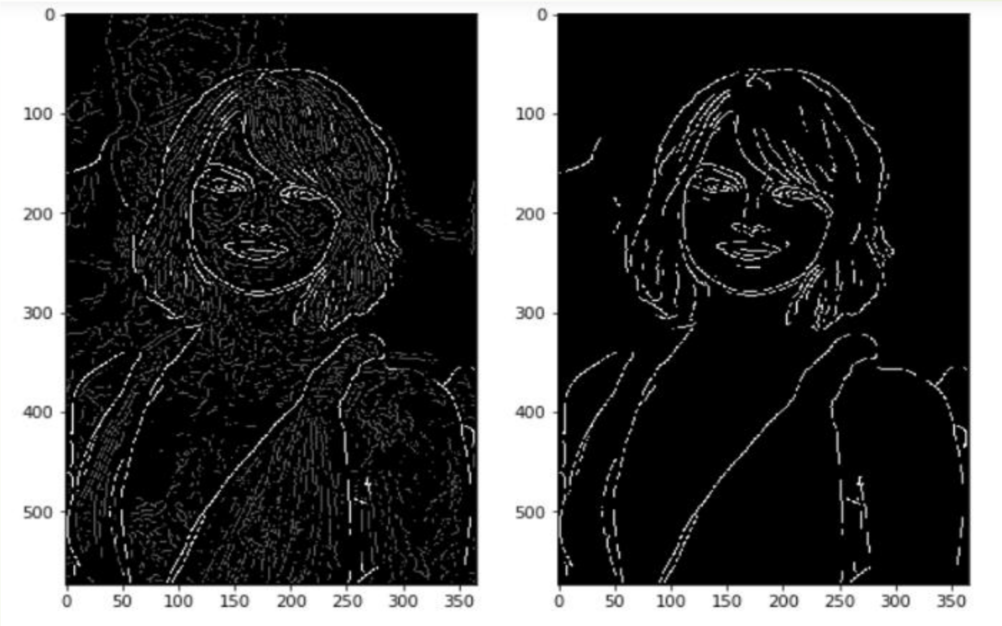


Non-Max Suppression image (left) — Threshold result (right): weak pixels in gray and strong ones in white

Edges

Canny Edge Detection

- Steps 5: Edge Tracking by Hysteresis
- It determines which weak edges are actual edges.
- To do this, it performs an edge tracking algorithm.
- Weak edges that are connected to strong edges will be actual/real edges.
- Weak edges that are not connected to strong edges will be removed.
- Finalize the detection of edges by suppressing all the other edges that are weak and not connected to strong edges.



Results of hysteresis process

Edges

Application: **Finger-print Recognition**

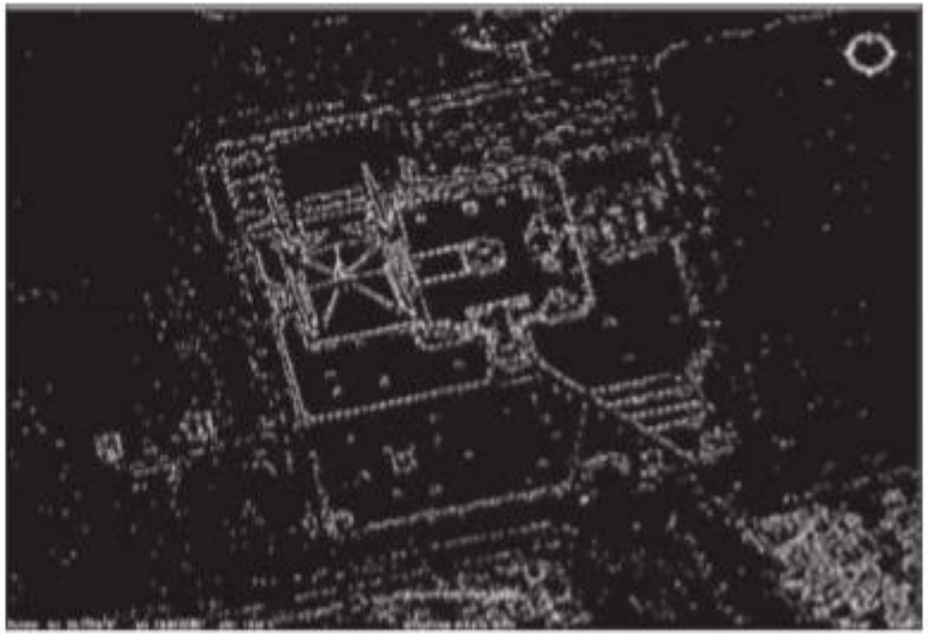
- At present finger-print recognition has been used widely such as in mobile phones.
- Edge detection techniques enhance the quality of image and cause the improvement in the image recognition.

Edges

Application: **Satellite Image**

- The edge map generated by bilateral filtering-based edge detection contain more accurate and well localized edge map.
- It also suppresses noise and produces more realistic edge map.

Satellite image



Canny based edge map of satellite image

Edges

Application: Robotics vision

- The present and near future main application areas of edge detection are robotics vision (e.g. in self-driving vehicles).
- Set the steering wheel angle based on picture of road.

Original image

Detected image

Edges

Application: **Medical Science**

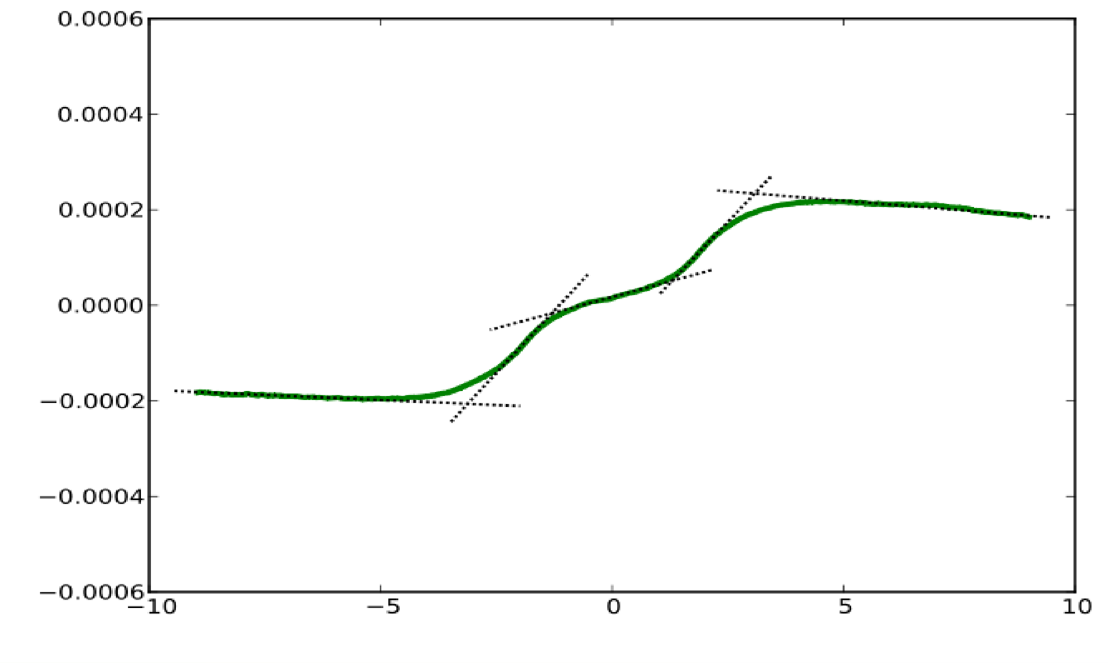
- Another important area being developed which of course involves edge detection is finding “pathological” objects in medical images such as tumors.
- Medical image edge detection is an important method in the recognition of the human organs.
- It is an important pre-processing step in 3D reconstruction such as the reconstruction of brain images.

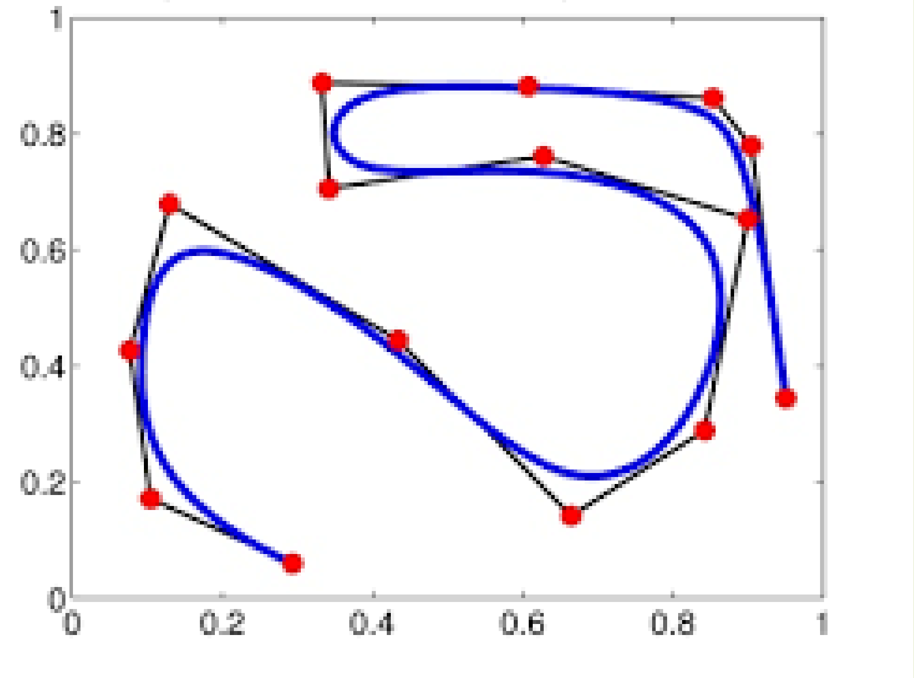
Original MRI Scan | from Sobel operator | from Canny operator

Lines

Successive Approximation

- It is preferable to approximate such a curve with a simpler representation, e.g., as a piecewise-linear polyline or as a B-spline curve (Farin 1996).





Lines

What is line detection?

- Lines in an image present another form of discontinuity
- It is worthwhile to consider them separately since they are very frequent
- Methods :
  - Use of Convolution Mask
  - Hough Transform

Lines

Line Detection using Convolution Mask

- In a convolution-based technique, the line detector operator consists of a convolution masks tuned to detect the presence of lines.
- We need to decide the orientation and width of line to be detected.
- Following masks are for single pixel wide lines in four directions.

−1	−1	−1
2	2	2
−1	−1	−1

(a) Horizontal mask

−1	2	−1
−1	2	−1
−1	2	−1

(b) Vertical mask

−1	−1	2
−1	2	−1
2	−1	−1

(c) Oblique (+45 degrees)

2	−1	−1
−1	2	−1
−1	−1	2

(d) Oblique (−45 degrees)

Lines

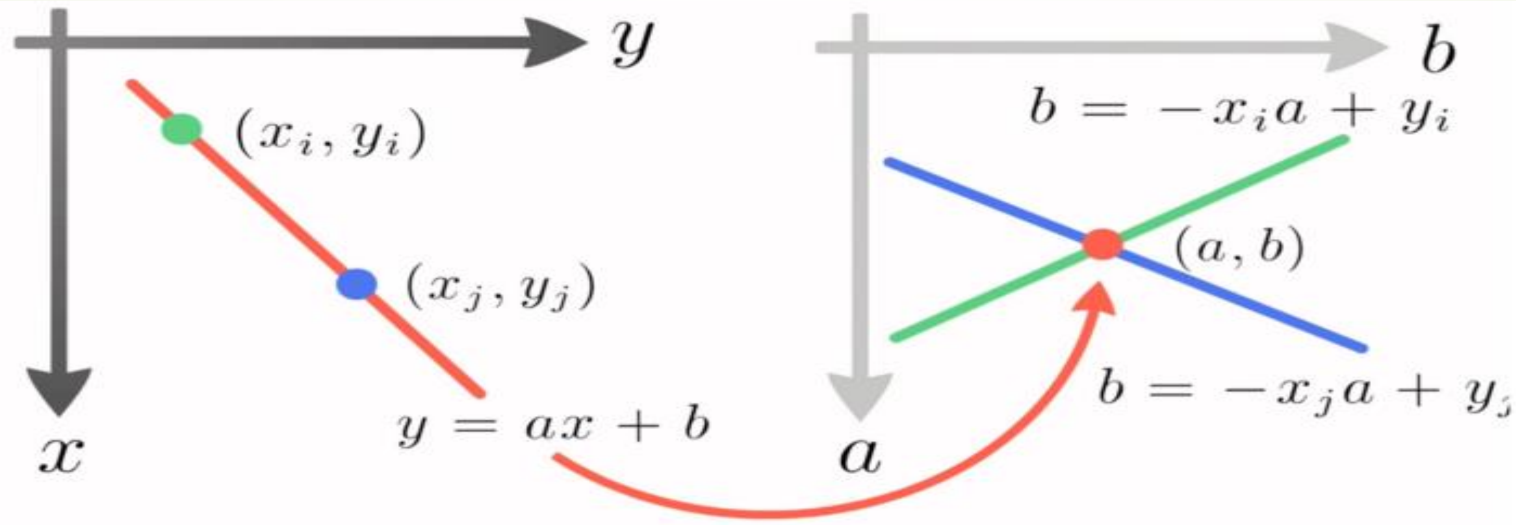
Hough Transform Technique

- In the Hough Transform algorithm, the Hough Space is used to determine whether a line exists in the edge image.
- The Hough transform algorithm requires an accumulator array
- whose dimension corresponds to the number of unknown parameters in the equation of the family of curves being sought.

Lines

Hough Space

- Connection between image (x,y) and Hough (a,b) spaces
  - A line in the image corresponds to a point in Hough space
  - To go from image space to Hough space:
    - given a set of points (x,y), find all (a,b) such that y = a x + b
  - What does a point (x1, y1) in the image space map to?
    - the solutions of b = - a x1 + y1 , this is a line in Hough space



Lines

Finding Straight-Line Segments

- $y = mx + b$ for straight lines does not work for vertical lines

  m : slope,   b : intercept
- To avoid this issue, a straight line is instead represented by a line called the normal line
- that passes through the origin and perpendicular to that straight line.

Lines

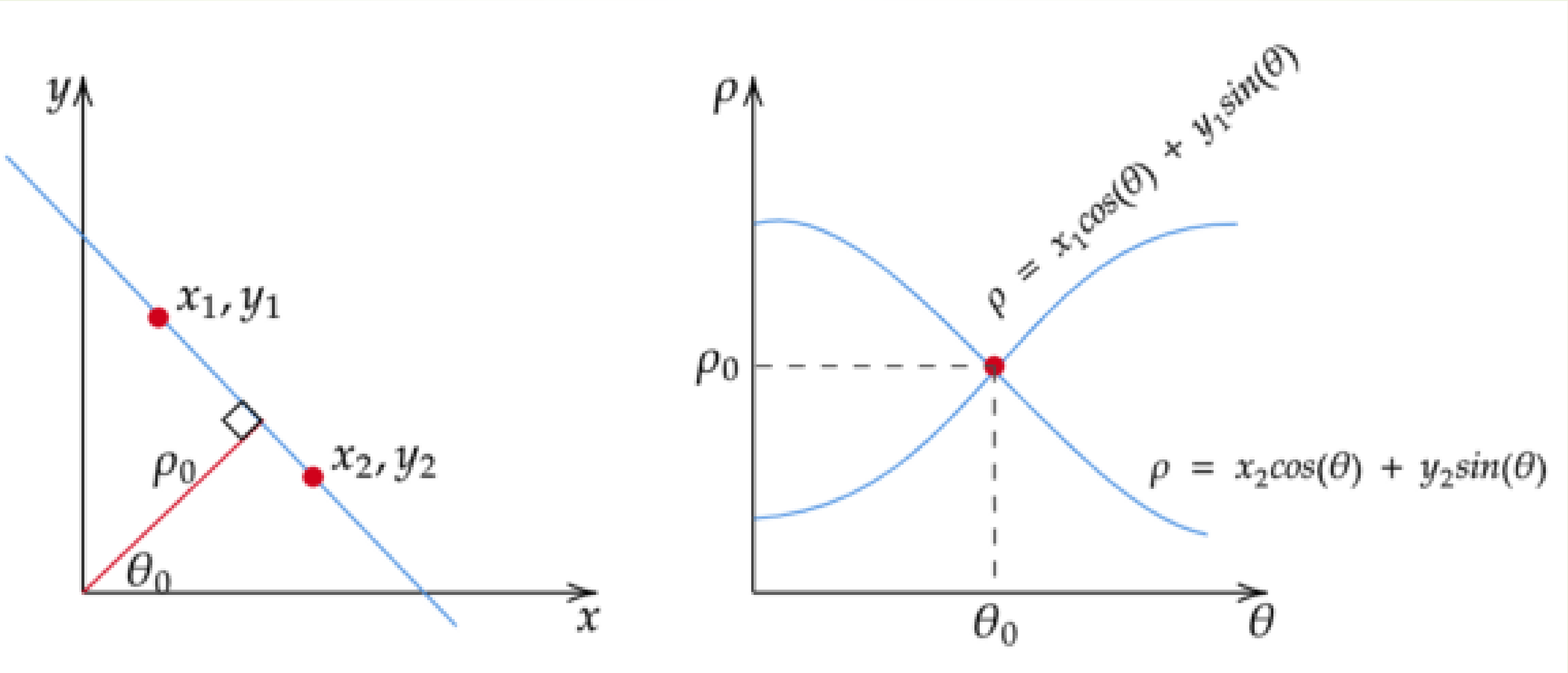
Parameterization

- Typically use a different parameterization

$$\rho = x\cos(\Theta) + y\sin(\Theta)$$

ρ is the perpendicular distance from the line to the origin

θ is the angle this perpendicular makes with the x axis



Lines

Hough Space Representation

- Instead of representing the Hough Space with the slope a and intercept b, it is now represented with ρ and θ
    - The horizontal axis are for the θ values and
    - The vertical axis are for the ρ values.
- The mapping of edge points onto the Hough Space works in a similar manner except that
    - An edge point (x_i, y_i) now generates a cosine curve in the Hough Space instead of a straight line.

Lines

Overview of the Algorithm

- If two edge points lay on the same line,
  - their corresponding cosine curves will intersect each other on a specific (ρ, θ) pair.
- The Hough Transform algorithm detects lines by
  - finding the (ρ, θ) pairs that has a number of intersections larger than a certain threshold.

Lines

Hough Transform Algorithm

1. Decide on the range of ρ and θ.
   - The range of θ is [0, 180] degrees and
   - ρ is [-d, d] where d is the length of the edge image's diagonal.
   - It is important to quantize the range of ρ and θ meaning there should be a finite number of possible values.
2. Create a 2D array called the accumulator representing:
   - The Hough Space with dimension (num_rhos, num_thetas) and
   - Initialize all its values to zero.
3. Perform edge detection on the original image with any edge detection algorithm.

Lines

Hough Transform Algorithm

4. For every pixel on the edge image,
   - check whether the pixel is an edge pixel.
   - If it is an edge pixel, loop through all possible values of θ, calculate the corresponding ρ,
   - find the θ and ρ index in the accumulator and increment the accumulator base on those index pairs.

```
Initialize H[ρ, θ]=0
for each edge point I[x,y] in the image
    for θ = 0 to 180
        ρ = x cos(Θ) + y sin(Θ)
        H[ρ, θ] += 1
```

5. Loop through all the values in the accumulator.
   - If the value is larger than a certain threshold,
     - get the ρ and θ index,
     - get the value of ρ and θ from the index pair
     - The detected line in the image is given by

$$\rho = x \cos(\Theta) + y \sin(\Theta)$$

Lines

Line Detection

Original Image